

SISTEMA FATEC · MODULO DE GRADE + MODULO DE FOLHA DE FREQUENCIA

Documentação Técnica do Banco de Dados

Backend único · PostgreSQL único · Projeto FATEC Zona Leste

Versão	Rascunho 1.1 para validação
Data	21/09/2026
Fontes consolidadas	Estrutura do Banco de Dados e Regras do Sistema; Documentação Técnica — Folha de Frequência; Documentação Técnica — Backend do Sistema de Grade; prompt de requisitos desta documentação

Convenções deste documento

[Definido] — consta nos documentos-fonte do projeto.

[Sugestão] — recomendação técnica desta documentação, ainda sem validação oficial.

A definir — ponto sem definição oficial; nada foi inventado para preenchê-lo.

A seção 38 lista as divergências encontradas entre as fontes e a decisão adotada em cada caso.

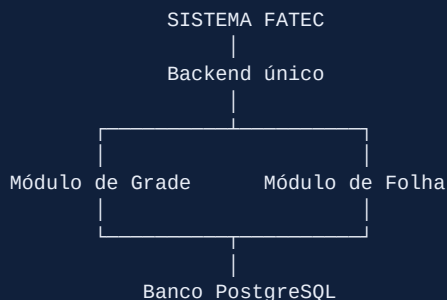
Sumário

1. Visão geral do banco de dados	4
2. Arquitetura do banco	4
3. Princípios da modelagem	5
4. Entidades principais	6
5. Estrutura das tabelas	6
6. Tipos de dados PostgreSQL	13
7. Chaves primárias	14
8. Chaves estrangeiras	14
9. Relacionamentos	14
10. Estrutura final proposta	15
11. MER — Modelo Entidade-Relacionamento	16
12. DER — Diagrama Entidade-Relacionamento (físico/lógico)	17
13. Formato dos diagramas	19
14. Regras de integridade	19
15. Regras de negócio no banco	20
16. Integração entre Grade e Folha	21
17. O que a Folha não deve fazer com a Grade	23
18. Calendário Acadêmico	23
19. Acesso e segurança	24
20. Autenticação	24
21. Autorização por perfil	25
22. Segurança do banco	25
23. Auditoria	26
24. Índices	26
25. Constraints	26
26. JPA / Hibernate	27
27. Migrations	29
28. Dados de exemplo	30
29. Exemplo de relacionamento completo	31
30. Consulta para a Folha	32
31. Separação de responsabilidades	32
32. Diagrama geral da arquitetura	33
33. Diagrama de segurança	34
34. Critérios de aceite do banco	34
35. Backlog / Issues do banco	35
36. Estrutura final do modelo	38
37. Regras importantes	39
38. Divergências entre as fontes e decisões adotadas	39
39. Verificação final de consistência	40

.....

1. Visão geral do banco de dados

O projeto FATEC possui **um único sistema**, com **um único backend** e **um único banco PostgreSQL**. Grade e Folha de Frequência são **módulos** desse sistema, não sistemas independentes.



- A **Grade** fornece as informações acadêmicas e de planejamento das aulas de cada professor em cada período letivo.
- A **Folha de Frequência** lê essas informações para preencher o documento oficial (DOCX). Ela **não** transforma a Grade em registro de presença.
- O banco tem duas camadas conceituais separadas:
 - **camada acadêmica:** PROFESSOR , CURSO , DISCIPLINA , PERIODO_LETIVO , GRADE , DISCIPLINA_DA_GRADE , AULA_DA_GRADE ;
 - **camada de segurança/acesso:** USUARIO .

O que o banco armazena e o que não armazena

Armazena	Não armazena
Professores, cursos, disciplinas, períodos letivos	Presença, falta, atraso, entrada/saída (não existe tabela de presença ou ponto)
Grades, disciplinas da grade, aulas (dia + horário)	O DOCX gerado (a recomendação D-15 da Folha é devolver para download, sem gravar no servidor)
Carga horária (na própria GRADE)	Datas do mês, dias da semana e domingos (são calculados , não persistidos)
Usuários, perfis, hash de senha	Slots da Folha (calculados a partir de hora_inicio pelo conversor de horários)
Datas de criação/atualização de GRADE e USUARIO	Calendário Acadêmico (armazenamento definitivo: A definir , ver seção 18)

2. Arquitetura do banco

2.1 Stack

Camada	Tecnologia
Linguagem	Java 21
Framework	Spring Boot
Persistência	Spring Data JPA + Hibernate
Banco de dados	PostgreSQL
Documentos	Apache POI (DOCX)
Validação	Jakarta Validation

2.2 Fluxo de dados (consultas)

Frontend → Controller → Service → Repository → PostgreSQL

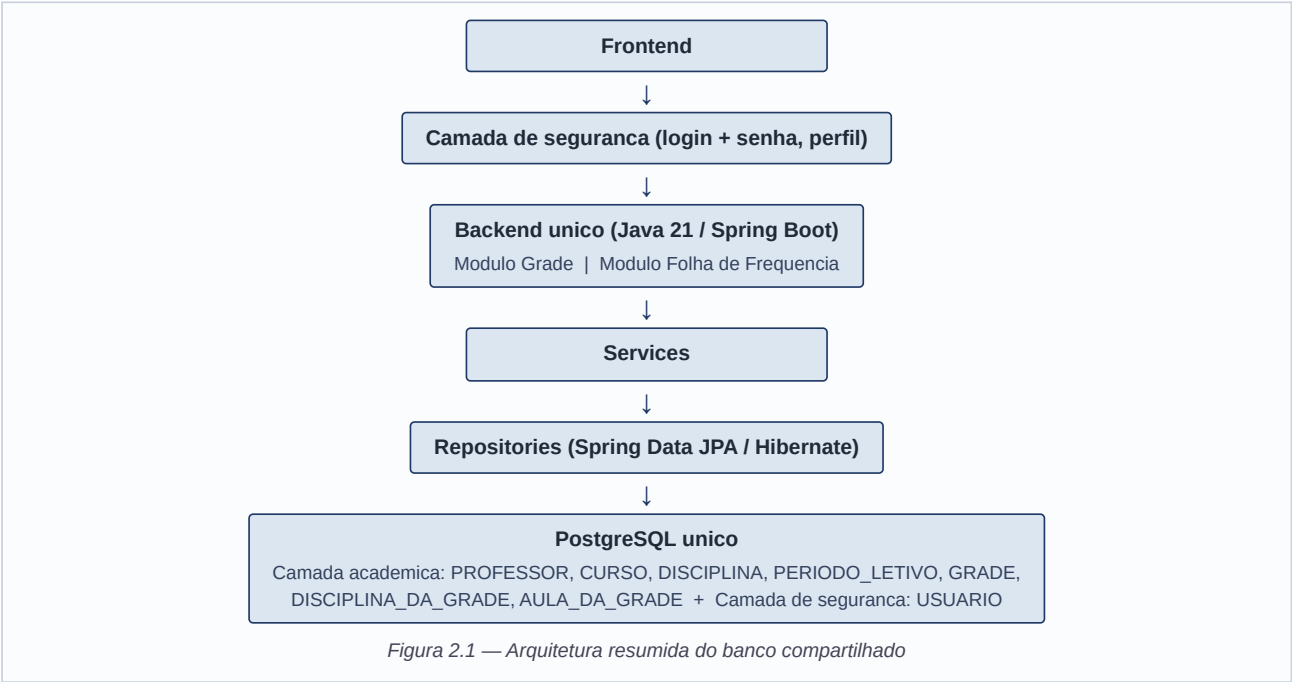
2.3 Fluxo de dados (documentos)

Frontend → Controller → FolhaFrequenciaService → Dados da Grade
→ Document Service → Apache POI → Modelo DOCX → Documento preenchido

Nomenclatura: o prompt original cita `FolhaPontoService`. A documentação da Folha (decisão **D-01**, ainda a confirmar) recomenda `FolhaFrequenciaService`, para coincidir com o título oficial do documento e não sugerir controle de ponto. Este documento adota **FolhaFrequenciaService**.

2.4 Por que um único banco

- **Fonte única de verdade:** a Folha lê a mesma Grade que o módulo de Grade mantém, sem cópia nem sincronização.
- **Sem duplicação:** não há segundo cadastro de professores nem de grades.
- **Consistência:** uma alteração na Grade aparece na próxima geração da Folha.



3. Princípios da modelagem

Princípio	Como é aplicado neste modelo
Normalização adequada	<code>DISCIPLINA</code> não possui <code>curso_id</code> : a mesma disciplina pode existir em vários cursos, e o vínculo é feito por <code>DISCIPLINA_DA_GRADE</code> .
Integridade referencial	Toda relação possui FK, com regra de exclusão explícita (seção 14).
Chaves primárias	Todas as 8 tabelas têm <code>id BIGINT</code> como PK.
Restrições	<code>UNIQUE</code> , <code>NOT NULL</code> e <code>CHECK</code> só onde há regra de negócio que os justifique (seção 25).
Tipos apropriados	<code>TIME</code> para horários, <code>DATE</code> para datas, <code>NUMERIC</code> para horas, <code>BOOLEAN</code> para flags (seção 6).

Princípio	Como é aplicado neste modelo
Sem duplicação	Sigla, código e nome ficam só em <code>DISCIPLINA</code> ; nome do professor só em <code>PROFESSOR</code> . Dados obtidos por relacionamento não são copiados.
Histórico	<code>PERIODO_LETIVO</code> + <code>GRADE</code> preservam Grades de períodos anteriores; exclusões usam <code>RESTRICT</code> .
Segurança	Senha só como hash; <code>ativo</code> bloqueia sem excluir; credenciais fora do código.
Rastreabilidade	<code>created_at</code> / <code>updated_at</code> em <code>GRADE</code> e <code>USUARIO</code> .
Manutenção	Migrations versionadas, nomes em <code>snake_case</code> e mapeamento JPA direto.

Redundância controlada [Definido]: `quantidade_aulas` (em `DISCIPLINA_DA_GRADE`) e a quantidade de linhas em `AULA_DA_GRADE` expressam a mesma ideia. A regra de negócio exige que coincidam. Essa é a única redundância intencional do modelo e é validada no `GradeService` .

Dados derivados que não são armazenados: slot do modelo DOCX, dia da semana de cada data do mês, domingos e o texto da sigla no documento. Todos são calculados na geração.

4. Entidades principais

#	Entidade	Camada	Função	Usada por
1	<code>PROFESSOR</code>	Acadêmica	Cadastro do docente	Grade, Folha
2	<code>CURSO</code>	Acadêmica	Cadastro do curso	Grade, Folha
3	<code>DISCIPLINA</code>	Acadêmica	Cadastro geral da matéria (código, sigla, nome)	Grade, Folha
4	<code>PERIODO_LETIVO</code>	Acadêmica	Semestre/ano em que uma Grade vale	Grade, Folha
5	<code>GRADE</code>	Acadêmica	Ensino de um professor em um período letivo, com carga horária	Grade, Folha
6	<code>DISCIPLINA_DA_GRADE</code>	Acadêmica	Disciplina dentro de uma Grade e de um curso	Grade, Folha
7	<code>AULA_DA_GRADE</code>	Acadêmica	Aula/bloco (dia + horário) de uma disciplina da Grade	Grade, Folha
8	<code>USUARIO</code>	Segurança	Acesso ao sistema (login, senha hash, perfil, ativo)	Backend inteiro

Essas oito entidades aparecem no MER (seção 11) e no DER (seção 12).

Entidades deliberadamente não criadas (não há requisito confirmado):

Candidata	Situação
<code>PRESENCA</code> / <code>PONTO</code>	Fora do escopo: o sistema prepara o documento, não controla frequência.
<code>TURMA</code>	Pendência da FATEC. Por ora, <code>DISCIPLINA_DA_GRADE</code> (curso + semestre do curso + período) é o equivalente funcional.
<code>ATIVIDADE_DA_GRADE</code> (V/D, código, percentual)	O significado ainda precisa ser confirmado; não é armazenado neste momento.
<code>CALENDARIO_ACADEMICO</code>	Armazenamento A definir (MVP sugerido na Folha: JSON versionado por semestre).
<code>AUDITORIA</code>	Sem requisito definido; possibilidade futura (seção 23).

5. Estrutura das tabelas

Legenda da coluna "Obrigatório":

- **Sim** — exigido por regra de negócio dos documentos-fonte ou por PK/FK.
- **Sim*** — sugestão técnica (o campo não faz sentido vazio), a validar.
- **Não** — pode ser nulo.
- **A definir** — obrigatoriedade não definida oficialmente. No DDL de referência fica nulável.

Os tipos e nomes de campo seguem o **modelo existente** (documento 1).

5.1 PROFESSOR

Finalidade: cadastro dos docentes. É usada pela `GRADE` (quem leciona) e pela Folha (bloco de identificação: `{{PROFESSOR}}`, `{{COD}}`, `{{MAT}}`, `{{RJ}}`).

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador interno do professor.
nome	VARCHAR(150)	—	Sim	Nome completo (origem de <code>{{PROFESSOR}}</code> , placeholder obrigatório na Folha).
codigo	VARCHAR(30)	—	Não	Código do professor, quando existir. Origem de <code>{{COD}}</code> na Folha. [Decisão adotada] campo incorporado ao modelo (antes era proposta em análise); coluna nulável, pois nem todo professor possui código.
cpf	VARCHAR(11)	UNIQUE	Sim	CPF, somente dígitos. Obrigatório e único por regra de negócio.
matricula	VARCHAR(30)	UNIQUE	Sim	Matrícula institucional. Obrigatória e única por regra de negócio (origem de <code>{{MAT}}</code>).
regime_contrato	VARCHAR(20)	—	Sim	DETERMINADO / INDETERMINADO / TEMPORARIO (valores iniciais, a confirmar). Nome do campo conforme o modelo existente .
regime_juridico	VARCHAR(20)	—	Não	Regime jurídico do professor, conforme citado no prompt de requisitos . [Decisão adotada] mantido lado a lado com <code>regime_contrato</code> a pedido; ver nota abaixo.
status	VARCHAR(20)	—	Sim	ATIVO / INATIVO / AFASTADO (valores iniciais, a confirmar).
titulacao	VARCHAR(30)	—	A definir	GRADUADO, ESPECIALISTA, MESTRE, DOUTOR, PÓS-DOUTOR (valores iniciais, a confirmar).

Nota sobre `regime_contrato` × `regime_juridico` [Sugestão]: o modelo existente (documento de referência do banco) usa `regime_contrato`; o prompt de requisitos usa `regime_juridico`. A pedido, **os dois campos foram mantidos** na tabela, em vez de escolher um só. Fica como pendência confirmar com a FATEC se representam a mesma informação (caso em que um dos dois se torna redundante e pode ser removido em uma migration futura) ou se são conceitos distintos (por exemplo, "regime jurídico" como vínculo estatutário/CLT e "regime de contrato" como prazo determinado/indeterminado) — **A definir**. O placeholder `{{RJ}}` da Folha pode ser satisfeito por qualquer um dos dois, a depender da definição.

Campo em análise que permanece fora do modelo:

Campo	Situação
<code>categoria</code>	Aparece no exemplo da Folha, mas não há <code>{{CATEGORIA}}</code> no modelo (D-09). Decisão: não entra no modelo. Só se justificaria se o DOCX oficial exibisse a categoria — o que ainda não foi confirmado.

- **Chave primária:** `id`.
- **Chaves estrangeiras:** nenhuma.
- **Restrições:** `UNIQUE(cpf)`, `UNIQUE(matricula)`.
- **Relacionamentos:** `PROFESSOR 1:N GRADE`.
- **Regras de negócio:** CPF e matrícula são obrigatórios e únicos; `regime_contrato` e `status` devem ser informados; `codigo` e `regime_juridico` são opcionais (nem todo professor os possui). Um professor pode ter várias Grades (uma por período, no mínimo). Para retirar alguém de uso, altera-se `status`; **não se exclui** o professor, pois isso destruiria o histórico das Grades (seção 14).
- **Nota de proteção de dados [Sugestão]:** o CPF é dado pessoal. A Folha não o utiliza, portanto ele **não deve** ser exposto em DTOs de consulta nem em logs.

5.2 CURSO

Finalidade: cadastro dos cursos da unidade. Associa-se às disciplinas **dentro de uma Grade**, via `DISCIPLINA_DA_GRADE`.

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador do curso.
nome	VARCHAR(150)	—	Sim*	Nome do curso (ex.: Análise e Desenvolvimento de Sistemas).
turno	VARCHAR(20)	—	A definir	Turno predominante do curso.
unidade	VARCHAR(100)	—	A definir	Unidade (por ora, apenas FATEC Zona Leste).

- **Campo `codigo`** : o código oficial do curso **ainda não foi definido** (documento 1, seções 5.2 e 12). **A definir**. Ele não consta na tabela, e o exemplo `codigo VARCHAR(...)` do prompt não deve virar coluna antes dessa definição.
- **Chave primária:** `id` . **FK:** nenhuma.
- **Relacionamentos:** `CURSO 1:N DISCIPLINA_DA_GRADE` .
- **Regras de negócio:** uma disciplina dentro de uma Grade pode estar associada a um curso; o mesmo curso aparece em muitas Grades.

5.3 DISCIPLINA

Finalidade: cadastro geral da matéria, independente de curso, grade ou período.

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador da disciplina.
codigo	VARCHAR(30)	—	Sim*	Código oficial da disciplina (ex.: <code>MAG004</code>).
sigla	VARCHAR(20)	—	Sim*	Sigla curta , usada visualmente na Grade Horária (ex.: <code>MAG</code>). A Folha retorna 422 se ela estiver ausente.
nome	VARCHAR(150)	—	Sim*	Nome completo (ex.: Álgebra Linear).

`codigo` e `sigla` NÃO são a mesma informação:

Campo	Papel	Exemplo
<code>codigo</code>	Identificação oficial completa	<code>MAG004</code>
<code>sigla</code>	Identificação curta; é o que aparece nas células da Grade Horária do DOCX	<code>MAG</code>
<code>nome</code>	Nome completo	<code>Álgebra Linear</code>

- **Sem `curso_id`** : a mesma disciplina pode aparecer em mais de um curso (ex.: Banco de Dados no DSM e no ADS). O vínculo com o curso é feito por `DISCIPLINA_DA_GRADE` .
- **Relacionamentos:** `DISCIPLINA 1:N DISCIPLINA_DA_GRADE` .
- **Restrição:** `UNIQUE(codigo)` é possível, mas só se a regra "código oficial único" for confirmada (**A definir**, seção 25).

5.4 PERIODO_LETIVO

Finalidade: representa o período acadêmico ao qual uma Grade pertence. É a peça que **preserva o histórico**: sem ele, Grades de semestres diferentes se confundiriam (por exemplo, "Grade 2026/1" e "Grade 2026/2" do mesmo professor).

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador do período.
semestre	SMALLINT	—	Sim	1 ou 2.
ano	SMALLINT	—	Sim	Ano letivo (faixa válida: A definir).
data_inicio	DATE	—	A definir	Data de início do período.
data_fim	DATE	—	A definir	Data de término. Deve ser posterior a <code>data_inicio</code> .

- **Chave primária:** `id` . **FK:** nenhuma.
- **Relacionamentos:** `PERIODO_LETIVO 1:N GRADE` .

- **Regras de negócio:** `semestre` $\in \{1, 2\}$; `data_inicio` $<$ `data_fim`; um período tem várias Grades; Grades de períodos anteriores são mantidas.
- **Uso pela Folha:** `data_inicio` / `data_fim` permitem avisar quando o mês solicitado cai fora do período da Grade (D-13, recomendação: avisar sem bloquear).
- **Três conceitos que não podem ser confundidos:**

Informação	Exemplo	Onde fica
Período letivo	2º/2026	<code>PERIODO_LETIVO</code> (via <code>GRADE</code>)
Período/turno	MANHÃ	<code>DISCIPLINA_DA_GRADE.periodo</code>
Semestre do curso	3º	<code>DISCIPLINA_DA_GRADE.semestre_curso</code>

5.5 GRADE

Finalidade: organização acadêmica das aulas **de um professor em um período letivo**. Uma Grade pertence a exatamente 1 professor e 1 período. A **carga horária é armazenada na própria Grade**.

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador da Grade.
professor_id	BIGINT	FK → PROFESSOR.id	Sim	Professor responsável.
periodo_letivo_id	BIGINT	FK → PERIODO_LETIVO.id	Sim	Período letivo da Grade.
semanal	NUMERIC(6,2)	—	A definir	Total de horas semanais.
mensal	NUMERIC(6,2)	—	A definir	Total de horas mensais. A fórmula oficial semanal → mensal ainda precisa ser confirmada.
total	NUMERIC(6,2)	—	A definir	Total geral de horas.
created_at	TIMESTAMP	—	Sim*	Data e hora de criação da Grade.
updated_at	TIMESTAMP	—	Sim*	Data e hora da última atualização (permite apontar a Grade alterada mais recentemente).

O exemplo do prompt mostrava um único campo `carga_horaria` `INTEGER`. O modelo existente usa **três** campos `NUMERIC(6,2)` (`semanal`, `mensal`, `total`). Foi adotado o modelo existente (seção 38).

- **Chave primária:** `id`. **FKs:** `professor_id`, `periodo_letivo_id`.
- **Relacionamentos:** `PROFESSOR` 1:N `GRADE`; `PERIODO_LETIVO` 1:N `GRADE`; `GRADE` 1:N `DISCIPLINA_DA_GRADE`.
- **Regras de negócio:** toda Grade tem professor e período; um professor tem Grades em vários períodos; limitar a **uma Grade por professor/período** é uma regra a confirmar com a FATEC (**A definir**); valores de carga horária devem ser numéricos válidos.
- **A GRADE não registra presença.** Ela planeja aulas; não registra o que aconteceu.
- **Atenção a `updated_at` [Sugestão]:** alterar aulas ou disciplinas não atualiza `grade.updated_at` automaticamente. Se a "Grade mais recentemente atualizada" precisar refletir essas mudanças, o `GradeService` deve atualizar o campo na mesma transação.
- **V/D e Atividade** não são armazenados neste momento (significado a confirmar).

5.6 DISCIPLINA_DA_GRADE

Finalidade: representa uma disciplina **dentro de uma Grade específica e de um curso específico**. É uma tabela associativa com atributos próprios, uma das mais importantes do modelo.

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador do registro.
grade_id	BIGINT	FK → GRADE.id	Sim	Grade à qual pertence.

Campo	Tipo	Chave	Obrigatório	Descrição
disciplina_id	BIGINT	FK → DISCIPLINA.id	Sim	Disciplina utilizada.
curso_id	BIGINT	FK → CURSO.id	Sim*	Curso relacionado.
semestre_curso	SMALLINT	—	A definir	Semestre curricular da disciplina no curso (ex.: 3º).
periodo	VARCHAR(20)	—	A definir	Turno: Manhã, Tarde ou Noite.
quantidade_aulas	SMALLINT	—	Sim	Quantidade de aulas semanais. Deve ser maior que zero.
op	INTEGER	—	A definir	Código/identificador OP associado à disciplina naquela Grade. Significado, faixa e unicidade A definir .
status	VARCHAR(20)	—	Sim	Situação da disciplina na Grade (ex.: MINISTRANDO). Deve ser válido.

Por que a tabela existe:

1. Uma disciplina (**DISCIPLINA**) é um cadastro geral e pode aparecer em **várias Grades e cursos**.
 2. Curso, semestre do curso, turno, quantidade de aulas, OP e status só têm sentido **naquela disciplina naquela Grade**; por isso ficam aqui, e não em **DISCIPLINA**.
 3. O **op** fica aqui (e não em **DISCIPLINA**) porque está associado à disciplina dentro da Grade.
 4. O **status** é texto (e não booleano) para acomodar novos estados no futuro sem mudar a estrutura.
- **Chave primária:** **id**. **FKs:** **grade_id**, **disciplina_id**, **curso_id**.
 - **Relacionamentos:** **GRADE 1:N**, **DISCIPLINA 1:N**, **CURSO 1:N** → **DISCIPLINA_DA_GRADE**; **DISCIPLINA_DA_GRADE 1:N AULA_DA_GRADE**.
 - **Regras de negócio:** **quantidade_aulas > 0**; quantidade de linhas em **AULA_DA_GRADE** deve coincidir com **quantidade_aulas**; **status** válido.
 - **Turma:** não existe entidade **TURMA**. Curso + **semestre_curso** + **periodo** cumprem esse papel até a FATEC confirmar o conceito.
 - **Unicidade:** não se sugere **UNIQUE(grade_id, disciplina_id, curso_id)**. Sem o conceito de turma, a mesma disciplina pode legitimamente aparecer duas vezes na mesma Grade e curso (por exemplo, em turnos distintos). **A definir**.

5.7 AULA_DA_GRADE

Finalidade: representa as aulas/blocos de uma disciplina dentro da Grade. É a tabela que guarda os **horários** da Grade.

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador da aula.
disciplina_grade_id	BIGINT	FK → DISCIPLINA_DA_GRADE.id	Sim	Disciplina da Grade a que a aula pertence.
dia_semana	SMALLINT	—	Sim*	Dia da semana. Codificação numérica A definir (ver abaixo).
hora_inicio	TIME	—	Sim	Horário de início.
hora_fim	TIME	—	Sim	Horário de término. Deve ser maior que hora_inicio .

- **Nome da FK:** o modelo existente usa **disciplina_grade_id**, enquanto o prompt cita **disciplina_da_grade_id**. Foi mantido o nome do modelo existente (seção 38).
- **Codificação de **dia_semana** (SMALLINT):** o prompt ilustrava com texto (**TERÇA**), mas o modelo define **SMALLINT**. **[Sugestão]** usar ISO-8601 (1 = segunda ... 7 = domingo), que coincide com **java.time.DayOfWeek.getValue()**. Como o modelo oficial da Folha não tem aulas aos domingos, o uso prático é 1 a 6.
- **Como a tabela representa os horários:** cada linha é **uma aula individual** com dia, hora inicial e hora final. Com isso o horário completo da Grade é reconstruído ao ordenar as linhas por dia e hora.

- **Regras de horário [Definido]:** início e fim obrigatórios; fim > início; duração padrão de 50 minutos; sem sobreposição dentro da mesma Grade; intervalos entre aulas **não** são cadastrados (ex.: 08:00–08:50, 08:50–09:40, [intervalo 09:40–09:50], 09:50–10:40, 10:40–11:30).
- **Uma aula pode ocupar vários blocos/slots.** Um período contínuo como 13:00–18:20 cobre 6 slots de 50 minutos do modelo oficial (13:00, 13:50, 14:50, 15:40, 16:40 e 17:30). A forma de **guardar** isso no banco tem duas alternativas, discutidas a seguir.

Solução proposta para o bloco contínuo (resolve D-07b e esclarece D-08)

A Folha de Frequência (documento 2) deixou como **DECISÃO A CONFIRMAR (D-07b)** se um período contínuo de aulas deve virar uma linha por slot de 50 minutos ou uma única linha por bloco. Ela também registrou, em **D-08**, uma contradição aparente: o exemplo da Especificação mostra a sigla "DW" preenchendo os 6 slots de um dia, o que a regra "uma aula = um slot" não explicava, levando à recomendação de tratar aquele exemplo como "apenas ilustrativo".

Solução proposta [Sugestão]: `AULA_DA_GRADE` passa a guardar **um bloco contínuo por linha** — `hora_inicio` e `hora_fim` representam o horário real e completo da sessão de aula (ex.: 13:00–18:20), e não um slot isolado de 50 minutos. Isso não exige nenhuma coluna nova: é a mesma estrutura da seção 5.7, usada com um grão diferente.

	Modelo por slot (uma linha por aula de 50 min)	Modelo por bloco (uma linha por sessão contínua) — proposto
Linhas para 13:00–18:20	6	1
Risco de inconsistência	Alto: se alguém editar/apagar 1 das 6 linhas, o bloco fica incompleto sem que nada acuse o erro.	Baixo: o horário completo vive em um único registro.
Cadastro pelo usuário	Digitar 6 horários.	Digitar 1 horário ("terça, 13h às 18h20"), do jeito que o professor realmente informa a aula.
Preenchimento da Grade Horária no DOCX	Cada linha preenche 1 célula.	Uma linha pode preencher várias células — a <i>expansão</i> é feita no serviço (abaixo), não no banco.
Exemplo "DW" nos 6 slots (D-08)	Parecia contradizer a regra "1 aula = 1 slot".	Deixa de ser contradição: um bloco contínuo de fato ocupa vários slots seguidos; é o comportamento esperado, não um caso ilustrativo à parte.

Como funciona a expansão (na aplicação, não no banco) [Sugestão]:

1. o `GradeService / ConversorHorarioSlot` já mantém a tabela fixa de slots por turno (seção da Folha, "6.1 Slots fixos do modelo oficial");
2. para cada bloco (`hora_inicio` , `hora_fim`) de uma linha de `AULA_DA_GRADE` , o serviço percorre essa tabela e marca como "ocupados" todos os slots cujo horário de início cai dentro do intervalo [`hora_inicio` , `hora_fim`] ;
3. cada slot ocupado gera uma célula preenchida (com a sigla da disciplina) na Grade Horária do DOCX — é exatamente aí que a Folha usa a expansão, sem que o banco precise ter 6 linhas;
4. a mesma expansão **valida** `disciplina_da_grade.quantidade_aulas` : o número de slots encontrados deve coincidir com o valor cadastrado. Essa contagem **não pode ser feita por uma divisão simples de minutos** (`duração / 50`), porque os slots têm intervalos entre si (por exemplo, de 13:50 às 14:50 há 60 minutos entre os inícios dos slots 2º e 3º, não 50) — por isso a contagem percorre a tabela de slots, e não a duração bruta do bloco;
5. sobreposição de horários passa a ser verificada comparando **menos intervalos**, maiores e mais fáceis de auditar, em vez de 6 pares de horários por disciplina.

Esta é também a forma como o próprio documento de referência do banco já ilustra o dado de exemplo (seção 28 da fonte original): uma única linha com `horario_inicio: 13:00` e `horario_fim: 18:20` , e não seis linhas — a solução proposta apenas formaliza essa prática como a convenção oficial da tabela.

Exemplo 1 — Álgebra Linear (MAG), terça-feira, bloco da tarde:

Campo	Valor
<code>dia_semana</code>	2 (terça, ISO-8601)
<code>hora_inicio</code>	13:00
<code>hora_fim</code>	18:20

Campo	Valor
Slots ocupados na expansão	1º, 2º, 3º, 4º, 5º e 6º da tarde (13:00, 13:50, 14:50, 15:40, 16:40, 17:30)
<code>disciplina_da_grade.quantidade_aulas</code> validado	6

Exemplo 2 — Desenvolvimento Web (DW), quarta-feira, bloco da noite (mesmo cenário de exemplo usado no documento de referência do banco, seção 4):

Campo	Valor
<code>dia_semana</code>	3 (quarta, ISO-8601)
<code>hora_inicio</code>	19:20
<code>hora_fim</code>	21:00
Slots ocupados na expansão	1º e 2º da noite (19:20 e 20:10 — o 3º slot da noite, 21:10, fica fora do intervalo)
<code>disciplina_da_grade.quantidade_aulas</code> validado	2

Em ambos os casos, **uma única linha em AULA_DA_GRADE** basta, e o resultado da expansão bate exatamente com os dados de exemplo já usados nos documentos-fonte.

Nota: esta é uma recomendação técnica desta documentação, não uma decisão já validada pela FATEC. Se a equipe preferir manter o modelo por slot (uma linha por aula de 50 minutos, como no restante do documento), a estrutura da tabela não muda — muda apenas a granularidade das linhas gravadas, uma decisão de uso e não de esquema.

5.8 USUARIO

Finalidade: controle de acesso ao sistema. Pertence à **camada de segurança**, separada das entidades acadêmicas. Nenhuma FK liga `USUARIO` a `GRADE`, `PROFESSOR` etc.

Campo	Tipo	Chave	Obrigatório	Descrição
id	BIGINT	PK	Sim	Identificador do usuário.
nome	VARCHAR(150)	—	A definir	Nome do usuário.
login	VARCHAR(50)	UNIQUE	Sim	Login usado na autenticação.
senha_hash	VARCHAR(255)	—	Sim	Senha armazenada somente como hash . Nunca em texto puro.
perfil	VARCHAR(30)	—	Sim	<code>RESPONSAVEL</code> , <code>TI</code> ou <code>ACOMPANHAMENTO</code> (perfis iniciais).
ativo	BOOLEAN	—	Sim*	Permite bloquear o acesso sem excluir o usuário.
created_at	TIMESTAMP	—	Sim*	Data de criação.
updated_at	TIMESTAMP	—	Sim*	Data da última atualização.

Perfis [Definido]:

Perfil	Finalidade
<code>RESPONSAVEL</code>	Responsável pelo cadastro e impressão dos documentos.
<code>TI</code>	Equipe de TI da FATEC ZL, para possíveis correções e manutenção.
<code>ACOMPANHAMENTO</code>	Acesso destinado ao acompanhamento do projeto.

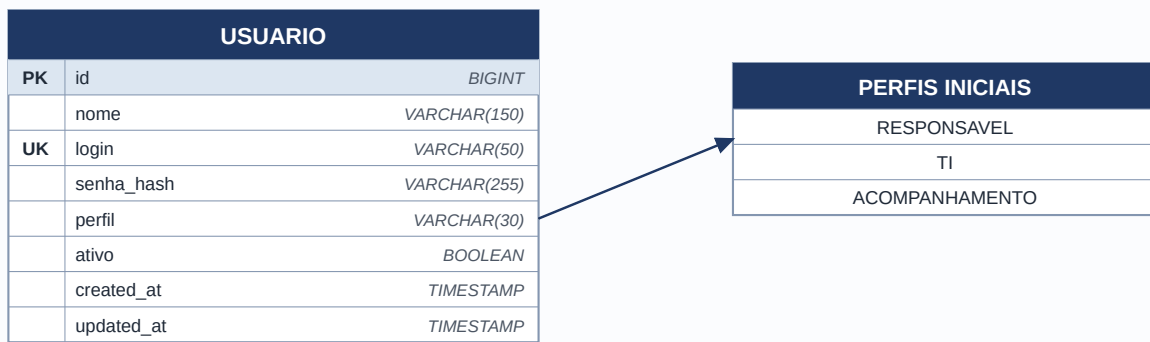


Figura 5.8.1 — Estrutura de USUARIO e perfis de acesso

- **Não há login de professor nesta fase.** O acesso é restrito a usuários autorizados.
- **Autenticação por login + senha.** A senha nunca é armazenada em texto puro; o campo é `senha_hash`.
- **ativo** : bloqueia o acesso sem apagar o registro (preserva a rastreabilidade).
- **created_at / updated_at** : auxiliam no controle e na auditoria dos usuários.
- **Chave primária:** `id`. **FKs:** nenhuma. **Restrições:** `UNIQUE(login)`.
- **Permissões exatas por perfil:** A definir (seção 21).

6. Tipos de dados PostgreSQL

Tipo	Quando usar	Onde aparece no modelo
BIGINT	Identificadores (PK/FK). Comporta grande volume e é o mapeamento natural de <code>Long</code> no Java.	Todos os <code>id</code> e <code>*_id</code>
INTEGER	Inteiros comuns sem restrição de faixa estreita.	<code>disciplina_da_grade.op</code>
SMALLINT	Inteiros pequenos e de domínio limitado.	<code>semestre</code> , <code>ano</code> , <code>semestre_curso</code> , <code>quantidade_aulas</code> , <code>dia_semana</code>
NUMERIC(6,2)	Valores decimais exatos (horas). Evita o erro de arredondamento de <code>FLOAT</code> .	<code>grade.semanal</code> , <code>mensal</code> , <code>total</code>
VARCHAR(n)	Texto de tamanho limitado, com o limite refletindo a regra de negócio.	nomes, códigos, siglas, login, status, perfil
BOOLEAN	Verdadeiro/falso.	<code>usuario.ativo</code>
DATE	Data sem hora.	<code>periodo_letivo.data_inicio</code> , <code>data_fim</code>
TIME	Hora do dia sem data ("hora de parede"). É o tipo natural de <code>hora_inicio</code> e <code>hora_fim</code> , permitindo comparação direta (<code>hora_fim > hora_inicio</code>) e ordenação.	<code>aula_da_grade.hora_inicio</code> , <code>hora_fim</code>
TIMESTAMP	Data e hora de eventos.	<code>created_at</code> , <code>updated_at</code>

Observações:

- **VARCHAR não é usado "para tudo":** horários são `TIME`, datas são `DATE`, flags são `BOOLEAN`, horas são `NUMERIC`.
- **VARCHAR(11) para CPF:** guarda apenas dígitos, sem máscara. **[Sugestão]** validar o formato na aplicação.
- **VARCHAR(255) para senha_hash** : comporta os formatos usuais de hash (BCrypt, Argon2).
- **TIMESTAMP × TIMESTAMPTZ** : o modelo existente define `TIMESTAMP`, que foi mantido. Se o sistema vier a operar em vários fusos, revisar essa escolha.
- **Valores de domínio (status, regime_contrato, perfil, periodo)**: ficam como `VARCHAR` em vez de `ENUM` nativo, porque as listas ainda são "iniciais e a confirmar" e alterar um `ENUM` do PostgreSQL exige migração.

7. Chaves primárias

Toda entidade principal possui `id` `BIGINT PRIMARY KEY`.

Função da PK: identificar cada linha de forma única e imutável, servir de alvo das FKs e criar automaticamente um índice único.

Tabela	PK
PROFESSOR, CURSO, DISCIPLINA, PERIODO_LETIVO	<code>id</code>
GRADE, DISCIPLINA_DA_GRADE, AULA_DA_GRADE	<code>id</code>
USUARIO	<code>id</code>

Geração automática (JPA/Hibernate): no PostgreSQL, `BIGINT GENERATED BY DEFAULT AS IDENTITY` (ou `BIGSERIAL`) combina com `@GeneratedValue(strategy = GenerationType.IDENTITY)`. O banco gera o valor e o Hibernate o lê após o `INSERT`. Alternativa: `SEQUENCE`, que permite lotes de inserção maiores ([Sugestão], decidir junto com o time). O `id` é um identificador técnico, sem significado de negócio, e **nunca é alterado**.

8. Chaves estrangeiras

#	FK	Referência	Obrigatória	ON DELETE	Significado
1	<code>GRADE.professor_id</code>	<code>PROFESSOR.id</code>	Sim	RESTRICT	Professor responsável pela Grade.
2	<code>GRADE.periodo_letivo_id</code>	<code>PERIODO_LETIVO.id</code>	Sim	RESTRICT	Período letivo da Grade.
3	<code>DISCIPLINA_DA_GRADE.grade_id</code>	<code>GRADE.id</code>	Sim	RESTRICT	Grade à qual a disciplina pertence.
4	<code>DISCIPLINA_DA_GRADE.disciplina_id</code>	<code>DISCIPLINA.id</code>	Sim	RESTRICT	Disciplina utilizada.
5	<code>DISCIPLINA_DA_GRADE.curso_id</code>	<code>CURSO.id</code>	Sim*	RESTRICT	Curso relacionado.
6	<code>AULA_DA_GRADE.disciplina_grade_id</code>	<code>DISCIPLINA_DA_GRADE.id</code>	Sim	RESTRICT	Disciplina da Grade a que a aula pertence.

Integridade referencial: o PostgreSQL impede (a) gravar uma FK que aponte para uma linha inexistente e (b) excluir uma linha "pai" que ainda tenha "filhos" (com `RESTRICT`). Assim, é impossível ter uma Grade sem professor, uma aula sem disciplina da Grade, etc. As regras de exclusão estão na seção 14. `ON UPDATE` fica no padrão (`NO ACTION`), pois os `id` são imutáveis. `USUARIO` não participa de nenhuma FK.

9. Relacionamentos

#	Relacionamento	FK utilizada	Cardinalidade	Significado e regra de negócio
1	PROFESSOR → GRADE	<code>GRADE.professor_id</code>	1:N	Um professor possui várias Grades (uma por período, no mínimo); cada Grade tem um único professor.
2	PERIODO_LETIVO → GRADE	<code>GRADE.periodo_letivo_id</code>	1:N	Um período reúne várias Grades; cada Grade pertence a um só período. Preserva o histórico.

#	Relacionamento	FK utilizada	Cardinalidade	Significado e regra de negócio
3	GRADE → DISCIPLINA_DA_GRADE	DISCIPLINA_DA_GRADE.grade_id	1:N	Uma Grade possui várias disciplinas; cada associação pertence a uma Grade.
4	DISCIPLINA → DISCIPLINA_DA_GRADE	DISCIPLINA_DA_GRADE.disciplina_id	1:N	Uma disciplina pode estar em várias Grades e cursos; cada associação usa uma disciplina.
5	CURSO → DISCIPLINA_DA_GRADE	DISCIPLINA_DA_GRADE.curso_id	1:N	Um curso pode ter várias disciplinas em várias Grades; cada associação tem um curso.
6	DISCIPLINA_DA_GRADE → AULA_DA_GRADE	AULA_DA_GRADE.disciplina_grade_id	1:N	Uma disciplina da Grade possui várias aulas/blocos; cada aula pertence a uma disciplina da Grade.
7	USUARIO	—	—	Controla o acesso ao sistema (login/senha, perfil). Não possui FK para as tabelas acadêmicas.

Sobre a forma original "N:1": o documento original escreve `DISCIPLINA N:1 DISCIPLINA_DA_GRADE` e `CURSO N:1 DISCIPLINA_DA_GRADE`. Essa notação lê a relação do lado da tabela filha, ou seja, *muitos DISCIPLINA_DA_GRADE para uma DISCIPLINA*. Semanticamente é o mesmo que `DISCIPLINA 1:N DISCIPLINA_DA_GRADE` ("uma disciplina pode estar associada a várias ocorrências de DISCIPLINA_DA_GRADE"). A informação original não se perde; apenas a cardinalidade é apresentada de forma consistente em todos os diagramas, sempre lida do lado "pai" para o "filho".

Participação:

- lado "1": obrigatório para o filho (toda FK é `NOT NULL`);
- lado "N": opcional para o pai (um professor pode existir sem Grades, um curso sem disciplinas na Grade).

10. Estrutura final proposta

```

PROFESSOR
  ↓
GRADE
  ↓
DISCIPLINA_DA_GRADE
  ↓
AULA_DA_GRADE

```

Relacionamentos complementares:

```

GRADE          → PERIODO_LETIVO
DISCIPLINA_DA_GRADE → DISCIPLINA
DISCIPLINA_DA_GRADE → CURSO
USUARIO        → ACESSO_AO_SISTEMA

```

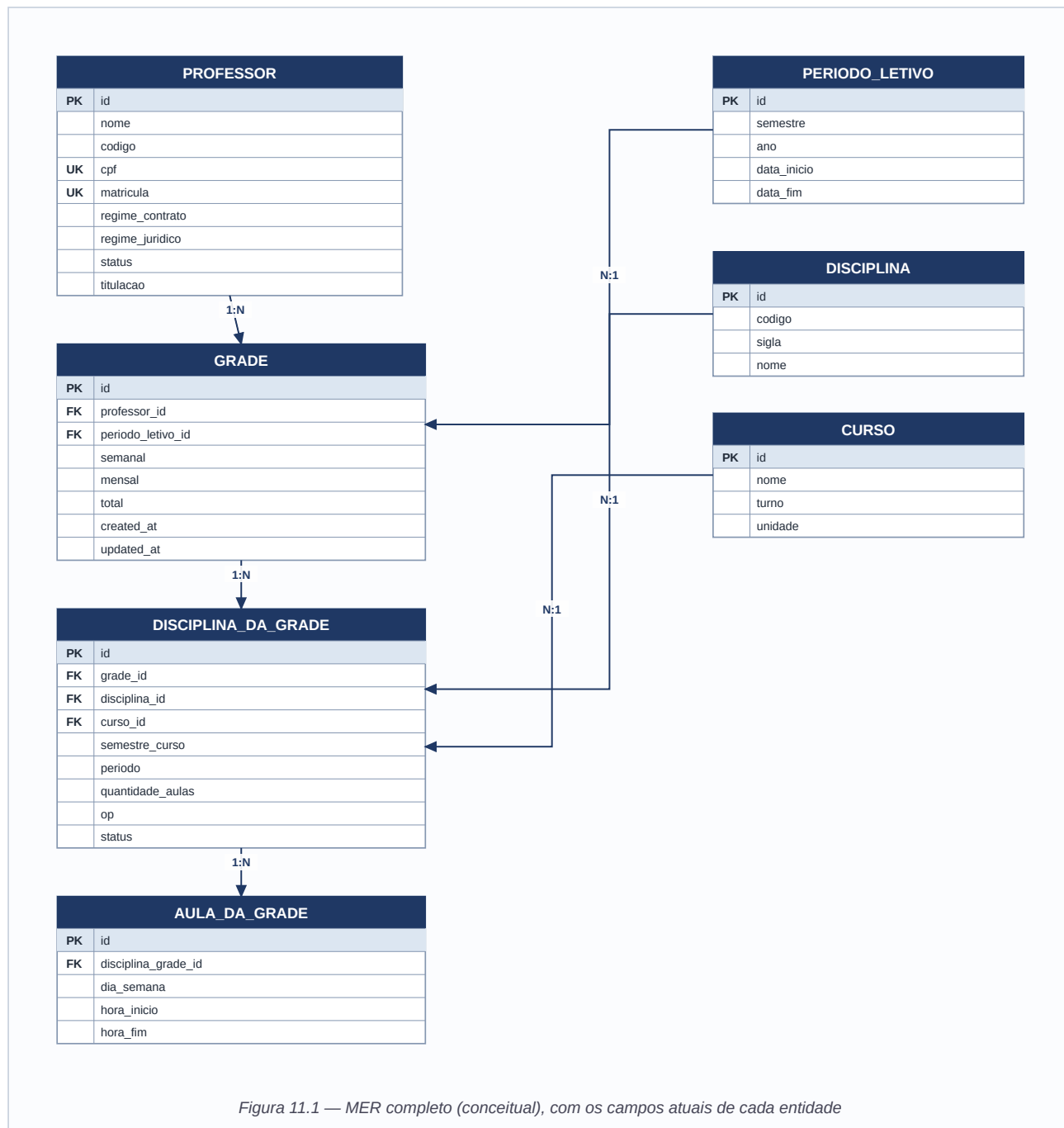
Explicação detalhada:

- Eixo principal.** `PROFESSOR → GRADE → DISCIPLINA_DA_GRADE → AULA_DA_GRADE` descreve o caminho do dado acadêmico: quem ensina, o que ensina e quando.
- GRADE é o ponto central.** Ela liga o professor ao período letivo e guarda a **carga horária** (`semanal`, `mensal`, `total`). Ter a carga horária aqui, e não espalhada nas disciplinas, evita duplicação e mantém o total num único lugar.
- PERIODO_LETIVO fica ao lado da Grade.** É o que separa "Grade 2026/1" de "Grade 2026/2".
- DISCIPLINA_DA_GRADE é o ponto de encontro** entre `GRADE`, `DISCIPLINA` e `CURSO`.
- AULA_DA_GRADE fica no final da cadeia.** Guarda o dia e o horário de cada aula.
- USUARIO fica fora da cadeia acadêmica.** Só controla quem pode usar o sistema.

Grade e Folha pertencem ao mesmo sistema, ao mesmo backend e ao mesmo banco.

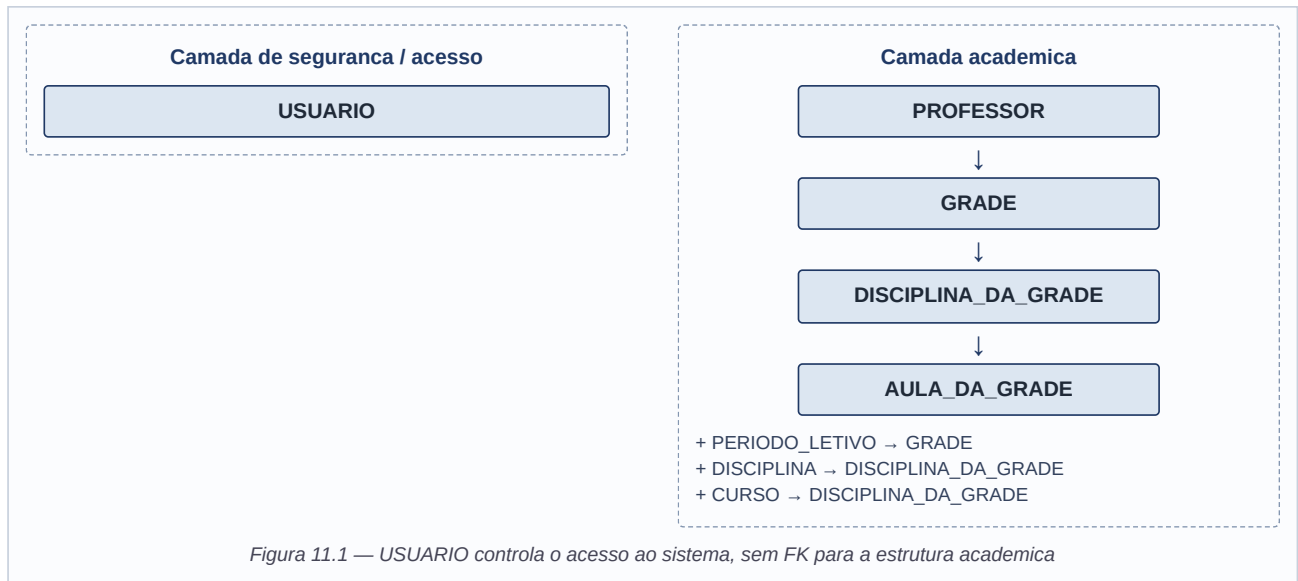
11. MER — Modelo Entidade-Relacionamento

O MER é conceitual: mostra as entidades, os atributos principais e as cardinalidades, sem detalhar tipos físicos. Nomes de campo são os do modelo existente.



Cardinalidades representadas: PROFESSOR 1:N GRADE · PERIODO_LETIVO 1:N GRADE · GRADE 1:N DISCIPLINA_DA_GRADE · DISCIPLINA 1:N DISCIPLINA_DA_GRADE · CURSO 1:N DISCIPLINA_DA_GRADE · DISCIPLINA_DA_GRADE 1:N AULA_DA_GRADE .

Sobre USUARIO no MER: ele aparece **isolado de propósito**. Não existe tabela de auditoria, então **não foi inventado** um relacionamento **USUARIO 1:N operações/acessos** apenas para conectá-lo. **USUARIO** pertence à camada de segurança/acesso, não à estrutura acadêmica da Grade:



12. DER — Diagrama Entidade-Relacionamento (físico/lógico)

O DER mostra as tabelas PostgreSQL com tipos, PK, FK e constraints relevantes (na coluna de comentário).

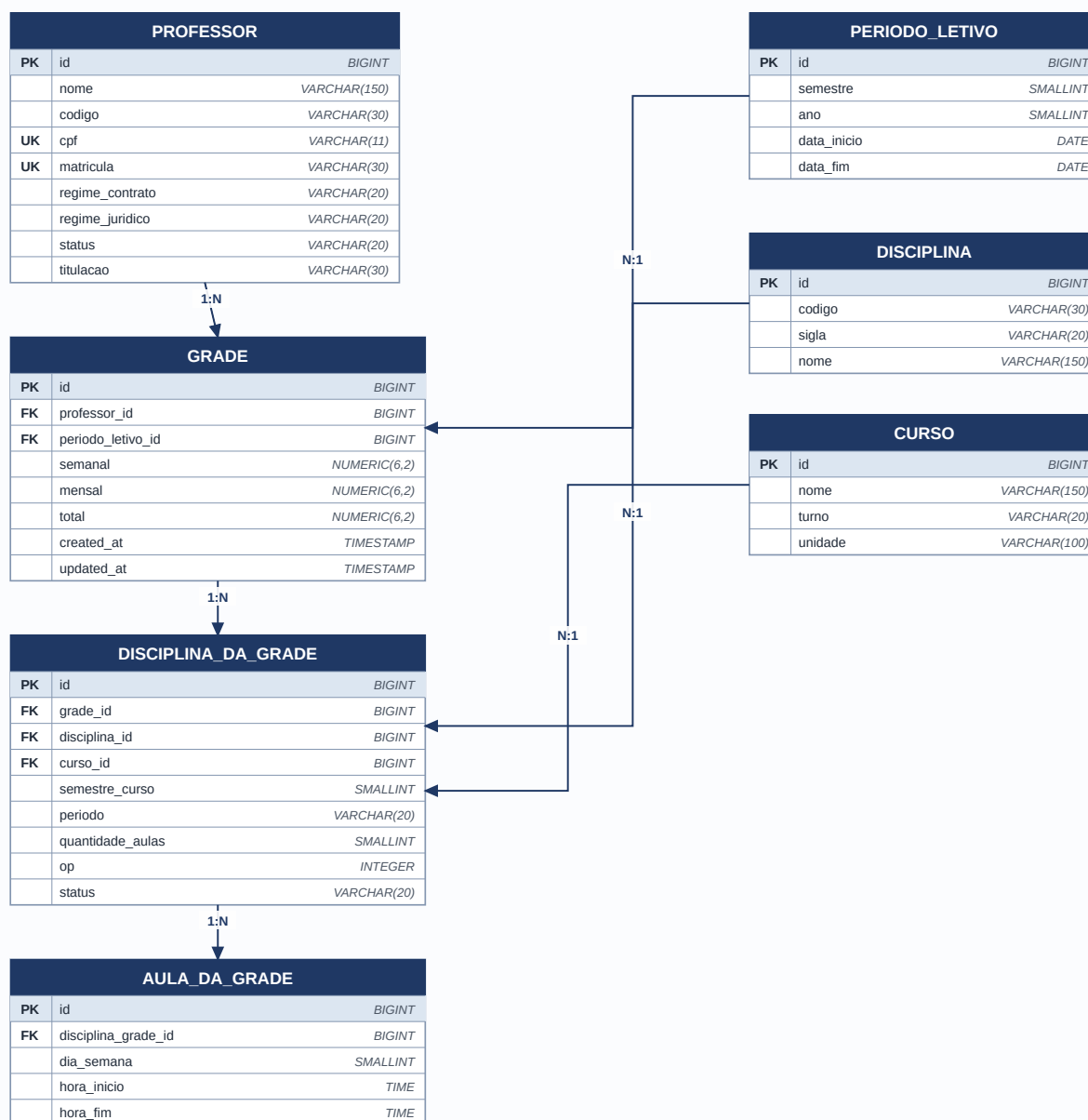


Figura 12.1 — DER completo (físico/lógico PostgreSQL), com tipos, PK e FK

Leitura visual (equivalente em texto):

```

PROFESSOR
  1
  |
  N
GRADE ————— N:1 ——— PERIODO_LETIVO
  |
  1
  |
  N
DISCIPLINA_DA_GRADE — N:1 — DISCIPLINA
                      | N:1 — CURSO
  |
  1
  |
  N
AULA_DA_GRADE

USUARIO (entidade de autenticação e autorização, sem FK para as demais)

```

13. Formato dos diagramas

Os diagramas usam **Mermaid** (`erDiagram` para MER/DER e `flowchart` para arquitetura e segurança). Regras seguidas:

- só existem campos que constam no modelo real; nenhum campo foi criado para "preencher" o diagrama;
- limitação do Mermaid: o tipo `NUMERIC(6,2)` é escrito como `NUMERIC` com `"6,2"` no comentário, pois vírgulas não são aceitas no tipo;
- as constraints mostradas nos comentários do DER são as descritas nas seções 14 e 25.

14. Regras de integridade

Visão geral (o detalhamento de `UNIQUE / CHECK` está na seção 25 e o dos índices na seção 24):

Tipo	Regras
PK	<code>id BIGINT</code> em todas as tabelas.
FK	6 chaves estrangeiras (seção 8), todas <code>NOT NULL</code> .
NOT NULL	Campos exigidos por regra de negócio; ver coluna "Obrigatório" na seção 5.
UNIQUE	<code>professor.cpf</code> , <code>professor.matricula</code> , <code>usuario.login</code> .
CHECK	Semestre 1 ou 2; <code>data_inicio < data_fim</code> ; <code>quantidade_aulas > 0</code> ; <code>hora_fim > hora_inicio</code> .
Valores padrão	[Sugestão] <code>created_at / updated_at = now()</code> ; <code>usuario.ativo = TRUE</code> .
Índices	Nas FKs (o PostgreSQL não as indexa sozinho); ver seção 24.

14.1 Regras de exclusão (ON DELETE)

Princípio: **nenhuma exclusão pode apagar histórico acadêmico sem uma regra definida**. Por isso **não se aplica CASCADE automaticamente**.

Opção	O que faz	Impacto no modelo	Decisão
<code>ON DELETE CASCADE</code>	Apaga os filhos junto com o pai.	Excluir um professor apagaria suas Grades, disciplinas e aulas, ou seja, o histórico inteiro, com um único comando. Perigoso.	Não usar como padrão.
<code>ON DELETE RESTRICT</code>	Impede excluir o pai se houver filhos.	O histórico fica protegido; a exclusão só ocorre quando não há dependentes.	Padrão adotado em todas as FKs.
<code>ON DELETE SET NULL</code>	Zera a FK do filho.	Todas as FKs são obrigatórias (<code>NOT NULL</code>), então <code>SET NULL</code> é inaplicável. Uma Grade sem professor seria dado órfão.	Não usar .

Aplicação por relação:

FK	Regra	Justificativa e impacto
GRADE.professor_id	RESTRICT	Não se exclui um professor com Grades. Para tirá-lo de uso, altera-se <code>status</code> (<code>INATIVO</code> / <code>AFASTADO</code>).
GRADE.periodo_letivo_id	RESTRICT	Um período com Grades não pode desaparecer.
DISCIPLINA_DA_GRADE.grade_id	RESTRICT	Uma Grade com disciplinas não é apagada por acidente. Uma remoção deliberada exige operação explícita no <code>Service</code> (aulas → disciplinas da grade → grade, numa transação) e permissão adequada. A regra oficial de "quando se pode excluir uma Grade" é A definir (pendência "regras de manutenção do histórico").
DISCIPLINA_DA_GRADE.disciplina_id	RESTRICT	Uma disciplina em uso não pode ser apagada.
DISCIPLINA_DA_GRADE.curso_id	RESTRICT	Um curso em uso não pode ser apagado.
AULA_DA_GRADE.disciplina_grade_id	RESTRICT (padrão)	<code>CASCADE</code> foi avaliado aqui porque a aula não tem sentido sem a disciplina da Grade. Foi descartado como padrão porque removeria horários silenciosamente. Na edição normal de uma Grade, o <code>Service</code> remove as aulas explicitamente , na mesma transação. A definir : o time pode optar por <code>CASCADE</code> apenas nesta relação, se preferir.
USUARIO	—	Não é excluído para bloquear acesso; usa-se <code>ativo = false</code> .

No JPA, os `@OneToMany` **não devem** usar `CascadeType.REMOVE` / `orphanRemoval` de forma automática (ver seção 26).

15. Regras de negócio no banco

Área	Regra	Onde é garantida	Origem
Professor	Um professor pode possuir várias Grades.	FK <code>GRADE.professor_id</code> (sem unicidade)	Definido
Professor	CPF e matrícula obrigatórios e únicos.	<code>NOT NULL + UNIQUE</code>	Definido
Professor	Regime e status devem ser informados.	<code>NOT NULL</code> (+ validação de valores na aplicação)	Definido
Período	Uma Grade pertence a um período letivo.	FK <code>NOT NULL</code>	Definido
Período	Semestre 1 ou 2; <code>data_inicio < data_fim</code> .	<code>CHECK</code>	Definido
Disciplina	Uma disciplina pode aparecer em várias Grades.	Sem <code>curso_id</code> em <code>DISCIPLINA</code> ; N ocorrências em <code>DISCIPLINA_DA_GRADE</code>	Definido
Curso	Uma disciplina da Grade pode estar associada a um curso.	FK <code>DISCIPLINA_DA_GRADE.curso_id</code>	Definido
Aula	Uma disciplina da Grade possui várias aulas/blocos.	FK <code>AULA_DA_GRADE.disciplina_grade_id</code>	Definido
Aula	<code>quantidade_aulas > 0</code> .	<code>CHECK</code>	Definido
Aula	Nº de linhas de aula = <code>quantidade_aulas</code> .	<code>GradeService.validarGrade()</code>	Definido
Horário	Início e fim obrigatórios; fim > início.	<code>NOT NULL + CHECK</code>	Definido
Horário	Duração padrão de 50 min.	Validação de serviço (não <code>CHECK</code> , por causa da decisão D-07b sobre aulas geminadas)	Definido
Horário	Sem sobreposição na mesma Grade.	<code>GradeService</code> (envolve várias tabelas; ver seção 25)	Definido
Horário	Horários respeitam os horários válidos do sistema.	<code>GradeService</code> /conversor de slots	Definido
Carga horária	Totais na própria <code>GRADE</code> , numéricos válidos.	Colunas em <code>GRADE</code> (+ <code>CHECK >= 0</code> sugerido)	Definido

Área	Regra	Onde é garantida	Origem
Carga horária	Fórmula semanal → mensal.	—	A definir
Documento	Gerado a partir dos dados validados; sem digitação de dados já cadastrados.	Módulo Folha (leitura)	Definido
Folha	Usa dados da Grade para preencher o documento.	Consulta em leitura (seções 16 e 30)	Definido
Presença	O banco não vira controle de presença automaticamente.	Ausência de tabela de presença/ponto	Definido
Acesso	Não há login de professor nesta fase.	USUARIO sem FK para PROFESSOR	Definido

15.1 Histórico

- **Grades de períodos diferentes não se confundem:** cada Grade tem `professor_id` + `periodo_letivo_id`. Exemplo: João da Silva pode ter Grade 1º/2026, Grade 2º/2026, Grade 1º/2027.
- **Grades anteriores são mantidas:** nenhuma exclusão em cascata; `RESTRICT` em todas as FKs.
- **Cadastros são "desativados", não apagados:** professor → `status`; usuário → `ativo`.
- **Datas de controle:** `GRADE.created_at` / `updated_at` mostram quando a Grade foi criada e alterada.
- **Regras oficiais de manutenção do histórico:** A definir (pendência no documento 1, seção 12).
- **Auditoria de alterações acadêmicas** (quem mudou o quê): não existe; é possibilidade futura (seção 23).

16. Integração entre Grade e Folha

Esta é a parte central da integração. **Grade e Folha são módulos do mesmo sistema.** Não existem dois bancos nem dois backends.



A Folha **consulta** a Grade. Fluxo:

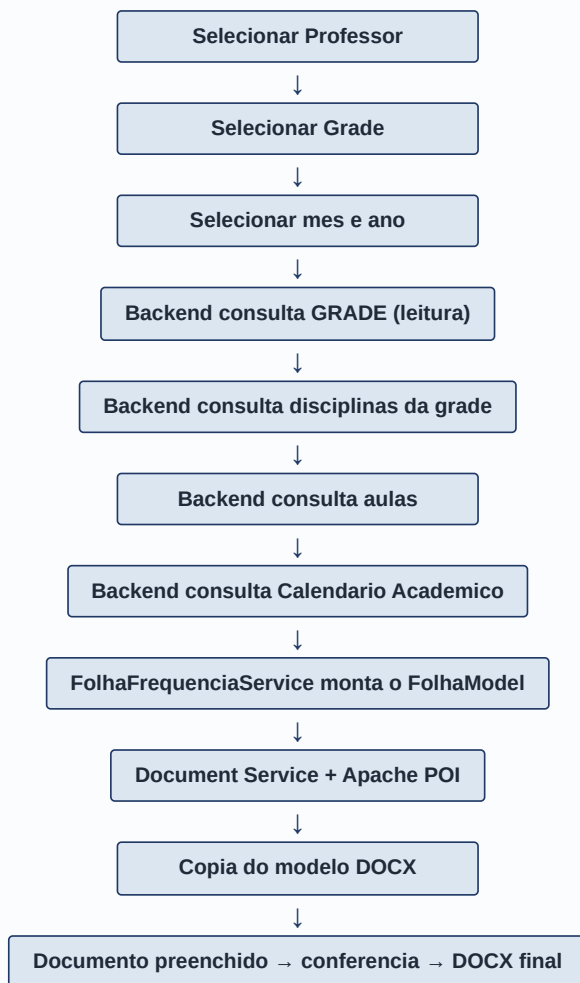


Figura 16.1 — Fluxo de integracao entre Grade e Folha (somente leitura)

16.1 Dados reaproveitados

Dado do documento	Origem no banco	Observação
Professor ({{PROFESSOR}})	GRADE.professor_id → PROFESSOR.nome	Placeholder obrigatório.
Matrícula ({{MAT}})	PROFESSOR.matricula	Se vazia, o documento mostra "—" (D-10).
Regime jurídico ({{RJ}})	PROFESSOR.regime_contrato	Confirmar se o texto do banco (DETERMINADO) coincide com o texto do documento ("Determinado").
Código ({{COD}})	PROFESSOR.codigo (campo proposto)	A definir (seção 5.1).
Categoria	—	A definir (D-09).
Disciplina / código / sigla / nome	DISCIPLINA_DA_GRADE.disciplina_id → DISCIPLINA	A Grade Horária do DOCX usa sempre a sigla , nunca o código.
Curso	DISCIPLINA_DA_GRADE.curso_id → CURSO.nome	Aparece na API/conferência.
Turno e semestre do curso	DISCIPLINA_DA_GRADE.periodo , semestre_curso	Equivalente funcional de "turma" (Alternativa A).
Dia e horário	AULA_DA_GRADE.dia_semana , hora_inicio , hora_fim	O slot do modelo é calculado a partir de hora_inicio .
Período acadêmico	GRADE.periodo_letivo_id → PERIODO_LETIVO	data_inicio / data_fim servem para o aviso "mês fora do período" (D-13).

Dado do documento	Origem no banco	Observação
Mês e ano do título	Informados pelo usuário	Não vêm do banco.
Datas, dias da semana, domingos	Calculados	Não são persistidos.
Dias sem aula	Calendário Acadêmico	Armazenamento A definir (seção 18).

Resumo:

```

GRADE           → professor
DISCIPLINA_DA_GRADE → disciplina
DISCIPLINA       → código / sigla / nome
CURSO            → curso
AULA_DA_GRADE    → dia e horário
PERIODO_LETIVO   → período acadêmico

```

Esses dados alimentam o módulo de Folha. **Nada disso é copiado** para outra tabela.

17. O que a Folha não deve fazer com a Grade

A geração da Folha é uma **operação de leitura** dos dados necessários da Grade:

```
GRADE —consulta→ dados para a Folha → DOCX
```

Não deve acontecer:

```
GRADE → Folha → alteração automática da Grade ✗
```

Motivos:

- evitar acoplamento indevido: gerar um documento não pode mudar o planejamento acadêmico;
- garantir que a Grade continue sendo a fonte de verdade;
- as edições feitas na conferência valem para o **documento**; não se assume que precisem voltar ao banco (documento da Folha, seção 9).

Medidas técnicas [Sugestão]:

- os métodos do módulo Folha só chamam consultas (`find*`), nunca `save / delete` , e rodam em `@Transactional(readOnly = true)` ;
- `FolhaFrequenciaDocxService` recebe apenas o `FolhaModel` pronto e **nunca** acessa banco nem outros Services;
- um teste de integração confere que, após gerar a Folha, `GRADE.updated_at` e as contagens de linhas permanecem iguais.

18. Calendário Acadêmico

- A **Grade** representa o planejamento das aulas. O **Calendário Acadêmico** informa datas que podem ser consideradas sem aula (feriados, emendas, recessos). A Folha usa as duas informações:

```
GRADE + CALENDÁRIO ACADÊMICO → FOLHA DE FREQUÊNCIA
```

- O calendário **não altera permanentemente a Grade**. A Grade Horária é semanal e não muda por causa de um feriado.
- Prioridade baixa:** o fluxo completo funciona sem o calendário (o `CalendarioService` devolve "nenhum evento" até a issue correspondente).
- Armazenamento definitivo: A definir** (decisão D-03). As fontes divergem: uma prevê tela administrativa ou importação CSV/Excel/JSON; a outra prevê PDF → revisão → JSON, sem sistema administrativo completo. A recomendação da

documentação da Folha para o MVP é um **JSON versionado por ano letivo/semestre**, lido pelo `CalendarioService` atrás de uma interface (assim, trocar "arquivo" por "banco" depois não afeta a geração do DOCX).

- **Consequência para este banco:** o calendário **não está entre as 8 entidades**, e nenhuma tabela foi criada. Se for decidido armazená-lo no PostgreSQL, será uma nova entidade, com sua própria decisão e migration.
- **Semântica de `semAula`:** neste documento, `semAula = true` significa **não há aula** (D-04).
- **Escopo da regra:** só importa o cruzamento onde **há aula na Grade** naquela data (D-06, recomendação).
- **Reposição não é implementada:** o calendário não gera nova aula na Grade.

19. Acesso e segurança

Toda a segurança de acesso usa a tabela `USUARIO`.

Perfil	Finalidade
<code>RESPONSAVEL</code>	Responsável pelo cadastro e impressão dos documentos.
<code>TI</code>	Equipe de TI da FATEC ZL, para possíveis correções e manutenção.
<code>ACOMPANHAMENTO</code>	Acesso destinado ao acompanhamento do projeto.

- **Não existe login de professor nesta fase.** O acesso é restrito aos usuários autorizados.
- O sistema autentica por **login + senha**.
- A senha **nunca** é armazenada em texto puro. Usa-se `senha_hash`.
- `ativo` permite **bloquear o acesso sem excluir** o usuário.
- `created_at` e `updated_at` auxiliam no controle e na auditoria dos usuários.

Ponto de atenção: o documento da Folha (D-16) trata autenticação e perfis como "não definidos" e "fora do escopo agora", enquanto o documento do banco já define `USUARIO`, os perfis e o login/senha. Este documento segue o **modelo de dados**: a tabela e as regras existem. Já a **implementação** da autenticação/autorização não consta nos backlogs de backend nem da Folha, por isso ela é tratada nas issues DB-15 e DB-16 (seção 35). A própria Folha recomenda tratar isso como requisito antes de expor a API fora da rede interna.

20. Autenticação

Autenticação responde "quem é você?" Fluxo:

```
Usuário informa login e senha
↓
Backend localiza USUARIO (por login)
↓
Verifica se ativo = true
↓
Compara a senha informada com senha_hash
↓
Autentica
↓
Verifica o perfil
↓
Autoriza a operação
```

Regras:

- Se `ativo = false`, o acesso é **bloqueado**. O usuário não é excluído para impedir o acesso.
- A comparação é feita sobre o **hash**. A senha original nunca é gravada nem registrada em log.
- **[Sugestão]** responder da mesma forma quando o login não existe, a senha está errada ou o usuário está inativo, para não revelar quais logins existem.

- **[Sugestão]** usar hash adaptativo com salt (BCrypt ou Argon2, via Spring Security `PasswordEncoder`). O algoritmo exato é **A definir**.
- **[Sugestão]** limitar tentativas de login e definir a política de expiração de sessão/token (**A definir**).

21. Autorização por perfil

Autorização responde "o que você pode fazer?" A tabela abaixo é uma **proposta** baseada nos perfis definidos. **Nenhuma permissão foi validada oficialmente**; não se deve assumir que os três perfis têm exatamente essas permissões.

Operação	RESPONSÁVEL	TI	ACOMPANHAMENTO
Consultar dados (professores, grades, disciplinas...)	Proposta: sim	Proposta: sim	Proposta: sim, somente leitura
Cadastrar informações (grades e dados acadêmicos)	Proposta: sim	A definir	Não proposto
Gerar documentos (Grade, Folha)	Proposta: sim	A definir	A definir
Imprimir documentos	Proposta: sim	A definir	A definir
Corrigir dados / manutenção / suporte técnico	A definir	Proposta: sim	Não proposto
Gerenciar <code>USUARIO</code> (criar, bloquear, trocar perfil)	A definir	A definir (candidato natural)	Não proposto
Acompanhar o andamento do projeto	A definir	A definir	Proposta: sim

Onde impor as regras [Sugestão]: no backend (Spring Security, por endpoint/método), nunca só no frontend. O banco não implementa perfis: o perfil é um dado do `USUARIO` interpretado pela aplicação.

22. Segurança do banco

Prática	Aplicação neste projeto
Senhas com hash	<code>senha_hash</code> ; nunca texto puro.
Conexão protegida	Conexão à base com TLS quando o banco não estiver na mesma máquina/rede isolada.
Credenciais fora do código	Usuário/senha do banco e demais <i>secrets</i> em variáveis de ambiente ou cofre de segredos, por perfil.
Nada de <i>secrets</i> no Git	Arquivos <code>.env</code> e credenciais em <code>.gitignore</code> ; <code>application.properties</code> só com placeholders (<code>\${DB_PASSWORD}</code>).
Menor privilégio	Usuário de aplicação com <code>SELECT/INSERT/UPDATE</code> (e <code>DELETE</code> só onde houver regra); sem <code>DROP / ALTER</code> . Um usuário separado, mais poderoso, executa as migrations.
Validação de entrada	Jakarta Validation nos DTOs; enums/valores válidos para <code>status</code> , <code>perfil</code> , <code>regime_contrato</code> .
Proteção dos endpoints	Autenticação + autorização por perfil (seções 20 e 21).
Controle de acesso	<code>USUARIO.ativo</code> , perfis, sessão/token com expiração.
Logs	Registrar tentativas de login e falhas de autorização; nunca senhas, hashes ou CPF.
Auditoria	<code>created_at / updated_at</code> (seção 23).
Backups	Rotina periódica de backup e teste de restauração (política e frequência A definir).
Migrations	Versionamento de esquema (seção 27).
SQL Injection	Uso de JPA e <i>queries</i> parametrizadas (<code>@Query</code> com parâmetros nomeados). Nunca concatenar entrada do usuário em SQL/JPL.
ddl-auto	<code>validate</code> (o Hibernate não cria nem altera tabelas).
Dados pessoais	CPF é dado pessoal: minimizar exposição em DTOs e logs; restringir acesso à tabela.

Não armazenar: senha em texto puro; credenciais do banco no código-fonte; *secrets* diretamente no Git.

23. Auditoria

- `created_at` e `updated_at` em `USUARIO` (e também em `GRADE`, conforme o modelo) permitem saber quando o registro nasceu e quando mudou pela última vez.
- Em `USUARIO`, isso ajuda a investigar quando uma conta foi criada, bloqueada (`ativo = false`) ou teve o perfil alterado.
- **[Sugestão]** manter `updated_at` automaticamente (JPA `@UpdateTimestamp` / `@PreUpdate` ou gatilho no banco), para que não dependa de cada `Service` lembrar de atualizá-lo.
- **Não existe tabela de auditoria**, porque não há requisito definido. Ela **não deve ser criada** sem requisito.
- **Possibilidade futura:** se a FATEC precisar saber quem alterou uma Grade e o que mudou, pode-se avaliar uma tabela de auditoria ou Hibernate Envers. Isso exigiria campos como "usuário responsável", que hoje não existem. Fica registrado apenas como possibilidade.

24. Índices

O PostgreSQL cria índices automaticamente para **PK** e **UNIQUE**, mas **não** para FKs. As FKs abaixo são usadas em `JOIN`s e filtros frequentes e são consultadas pelo banco a cada exclusão no lado "pai" (`RESTRICT`). Os índices são **[Sugestão]** e não se criam índices indiscriminadamente.

Índice	Colunas	Por que é útil	Origem
<code>uk_usuario_login</code> (automático)	<code>usuario.login</code>	Busca do login na autenticação. Já é coberto pelo <code>UNIQUE</code> ; não criar outro .	UNIQUE
<code>idx_grade_professor_id</code>	<code>grade.professor_id</code>	"Grades do professor" (<code>GET /api/professores/{id}/grades</code>); verificação de exclusão do professor.	FK
<code>idx_grade_periodo_letivo_id</code>	<code>grade.periodo_letivo_id</code>	"Grades do período"; verificação de exclusão do período.	FK
<code>idx_dg_grade_id</code>	<code>disciplina_da_grade.grade_id</code>	Carregar as disciplinas de uma Grade (consulta da Folha).	FK
<code>idx_dg_disciplina_id</code>	<code>disciplina_da_grade.disciplina_id</code>	Onde uma disciplina é usada; verificação de exclusão da disciplina.	FK
<code>idx_dg_curso_id</code>	<code>disciplina_da_grade.curso_id</code>	Disciplinas por curso; verificação de exclusão do curso.	FK
<code>idx_aula_disciplina_grade_id</code>	<code>aula_da_grade.disciplina_grade_id</code>	Carregar as aulas de uma disciplina da Grade (consulta da Folha).	FK

Não são indexados: `perfil`, `ativo`, `status` e `periodo` (poucos valores distintos, baixo ganho); e nomes/siglas, até haver busca por texto que justifique.

Índice composto opcional: se a regra "uma Grade por professor/período" for confirmada, um `UNIQUE(professor_id, periodo_letivo_id)` cobriria a busca "Grade do professor no período" (passo 4 do fluxo principal) e **substituiria** `idx_grade_professor_id`.

25. Constraints

Cada constraint abaixo tem uma regra de negócio que a justifica. Onde a regra ainda não foi confirmada, a constraint **não é aplicada**.

Constraint	Tabela.coluna(s)	Justificativa	Situação
UNIQUE	<code>usuario.login</code>	Dois usuários não podem ter o mesmo login.	Definido
UNIQUE	<code>professor.cpf</code>	CPF único por professor.	Definido
UNIQUE	<code>professor.matricula</code>	Matrícula única por professor.	Definido
NOT NULL	Campos obrigatórios da seção 5	Regras de negócio, PK e FK.	Definido / Sugestão (*)
CHECK	<code>periodo_letivo.semestre IN (1, 2)</code>	Semestre é 1 ou 2.	Definido
CHECK	<code>periodo_letivo.data_inicio < data_fim</code>	Data inicial anterior à final.	Definido
CHECK	<code>disciplina_da_grade.quantidade_aulas > 0</code>	Quantidade de aulas maior que zero.	Definido
CHECK	<code>aula_da_grade.hora_fim > hora_inicio</code>	Horário final maior que o inicial.	Definido
CHECK	<code>grade.semanal/mensal/total >= 0</code> (quando não nulos)	Horas negativas não fazem sentido.	Sugestão
CHECK	<code>professor.cpf ~ '^[0-9]{11}\$'</code>	CPF só com 11 dígitos, sem máscara.	Sugestão
UNIQUE	<code>periodo_letivo(ano, semestre)</code>	Evitar dois "2026/1" que dividiriam as Grades do mesmo semestre.	Sugestão
UNIQUE	<code>disciplina.codigo</code>	Código oficial único.	A definir (aguarda a confirmação dos códigos oficiais)
UNIQUE	<code>grade(professor_id, periodo_letivo_id)</code>	Uma Grade por professor/período.	A definir ("regra a confirmar com a FATEC")
CHECK	<code>aula_da_grade.dia_semana BETWEEN 1 AND 7</code>	Depende da codificação escolhida.	A definir
CHECK	<code>status, regime_contrato, perfil, periodo</code> em listas	As listas são "iniciais e a confirmar". Até lá, validar na aplicação.	A definir
CHECK	<code>ano</code> em faixa válida	Faixa "a definir".	A definir

Regras que o banco não consegue garantir sozinho (ficam no `GradeService`):

- **Sem sobreposição de horários na mesma Grade.** A `AULA_DA_GRADE` não guarda `grade_id`; a Grade só é alcançada por `DISCIPLINA_DA_GRADE`. Uma *exclusion constraint* exigiria copiar `grade_id` para `AULA_DA_GRADE` (duplicação que o modelo evita). **Decisão:** validar em `validarGrade()`.
- **Nº de aulas = quantidade_aulas.** Também envolve duas tabelas; só um gatilho garantiria isso, com custo e complexidade que não se justificam agora.
- **Duração de 50 minutos.** Não vira `CHECK` para não bloquear a decisão pendente sobre aulas geminadas (D-07b).

26. JPA / Hibernate

Mapeamento entre o PostgreSQL e as entidades Java. O foco é mostrar **como os relacionamentos do banco se expressam no código**; não é o código completo de todas as entidades.

Anotação	Função
<code>@Entity</code>	Marca a classe como entidade persistida.
<code>@Table(name = "grade")</code>	Nome da tabela (<code>snake_case</code>).
<code>@Id</code>	Chave primária.
<code>@GeneratedValue(strategy = IDENTITY)</code>	O PostgreSQL gera o <code>id</code> .
<code>@Column(name = "created_at", nullable = false)</code>	Mapeia coluna e obrigatoriedade.

Anotação	Função
<code>@ManyToOne(fetch = LAZY, optional = false)</code>	Lado "N" da relação (FK NOT NULL).
<code>@JoinColumn(name = "professor_id")</code>	Nome da coluna FK.
<code>@OneToMany(mappedBy = "...")</code>	Lado "1", navegação inversa (não cria coluna).

Mapeamento de tipos: `BIGINT` → `Long`, `SMALLINT` → `Short`, `INTEGER` → `Integer`, `NUMERIC(6,2)` → `BigDecimal`, `TIME` → `LocalTime`, `DATE` → `LocalDate`, `TIMESTAMP` → `LocalDateTime`, `BOOLEAN` → `Boolean`, e `VARCHAR` de domínio → `enum` com `@Enumerated(EnumType.STRING)`.

```
@Entity
@Table(name = "grade")
public class Grade {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "professor_id", nullable = false)
    private Professor professor;           // GRADE.professor_id -> PROFESSOR.id

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "periodo_letivo_id", nullable = false)
    private PeriodoLetivo periodoLetivo;   // GRADE.periodo_letivo_id -> PERIODO_LETIVO.id

    @Column(precision = 6, scale = 2) private BigDecimal semanal;
    @Column(precision = 6, scale = 2) private BigDecimal mensal;
    @Column(precision = 6, scale = 2) private BigDecimal total;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;
    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    // Lado inverso. SEM CascadeType.REMOVE e SEM orphanRemoval automáticos (seção 14.1)
    @OneToMany(mappedBy = "grade")
    private List<DisciplinaDaGrade> disciplinas = new ArrayList<>();
}
```

```

@Entity
@Table(name = "disciplina_da_grade")
public class DisciplinaDaGrade {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "grade_id", nullable = false)
    private Grade grade;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "disciplina_id", nullable = false)
    private Disciplina disciplina;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "curso_id", nullable = false)
    private Curso curso;

    @Column(name = "semestre_curso") private Short semestreCurso;
    @Column(length = 20) private String periodo; // Manhã | Tarde | Noite
    @Column(name = "quantidade_aulas", nullable = false)
    private Short quantidadeAulas;
    private Integer op; // significado: A definir
    @Column(length = 20, nullable = false) private String status; // ex.: MINISTRANDO

    @OneToMany(mappedBy = "disciplinaDaGrade")
    private List<AulaDaGrade> aulas = new ArrayList<>();
}

```

```

@Entity
@Table(name = "aula_da_grade")
public class AulaDaGrade {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "disciplina_grade_id", nullable = false) // nome da coluna no modelo existente
    private DisciplinaDaGrade disciplinaDaGrade;

    @Column(name = "dia_semana", nullable = false) private Short diaSemana; // codificação: A definir
    @Column(name = "hora_inicio", nullable = false) private LocalTime horaInicio;
    @Column(name = "hora_fim", nullable = false) private LocalTime horaFim;
}

```

Boas práticas:

- `spring.jpa.hibernate.ddl-auto=validate`: o Hibernate confere o esquema, mas não cria nem altera tabelas (o esquema é dono das migrations).
- **Não expor entidades na API**: usar DTOs (o documento do backend já define isso).
- `LAZY` nos `@ManyToOne` e `JOIN FETCH` nas consultas da Folha, para evitar o problema de N+1 consultas.
- `Usuario` deve ser mapeada com `@Column(unique = true)` em `login`, e nunca ter um getter de senha em texto.

27. Migrations

O banco precisa de **histórico das alterações estruturais**, para que qualquer ambiente (desenvolvimento, teste, produção) possa ser reconstruído e para que mudanças no esquema sejam revisadas como código.

Opção avaliada: Flyway (ou solução equivalente, como Liquibase). **Não se assume que o Flyway já esteja implementado**; isso é uma decisão a tomar.

Exemplo de versionamento:

Arquivo	Conteúdo
V1_criacao_tabelas.sql	Tabelas acadêmicas com PK, FK e constraints definidas.
V2_alteracao_grade.sql	Ajustes na Grade (ex.: colunas de carga horária, <code>created_at</code> / <code>updated_at</code>), se necessários.
V3_criacao_usuario.sql	Tabela <code>usuario</code> com <code>UNIQUE(login)</code> .
V4_indices.sql	[Sugestão] Índices da seção 24.
V5_professor_codigo.sql	[Sugestão, condicionada] Coluna <code>codigo</code> nulável em <code>professor</code> , se confirmada.

Observações:

- Cada migration é **imutável** depois de aplicada; correções entram em uma nova versão.
- Se o banco **já existe** (como afirmam os documentos de backend e Folha), é preciso um *baseline* (`baseline-on-migrate`) para que o Flyway parta do esquema atual, sem recriá-lo.
- Migrations rodam com um usuário com permissão de `DDL`; a aplicação usa um usuário de menor privilégio (seção 22).
- Adições (nova coluna nulável) são seguras; remover ou renomear colunas (por exemplo, alinhar `disciplina_grade_id` a `disciplina_da_grade_id`) exige planejamento e sincronização com as entidades JPA.

28. Dados de exemplo

Os dados abaixo são apenas ilustrativos. Servem para demonstrar a estrutura e **não devem ser tratados como dados reais** do sistema. Campos sem definição aparecem como "—" (nulo). A codificação de `dia_semana` usa ISO-8601 (1 = segunda ... 7 = domingo) somente como ilustração (**A definir**).

DISCIPLINA

id	codigo	sigla	nome
1	MAG004	MAG	Álgebra Linear

PROFESSOR

id	nome	cpf	matricula	regime_contrato	status	titulacao
1	Alexandre Falcão Sanseverino	00000000000 (fictício)	EXEMPLO-001 (fictícia)	DETERMINADO	ATIVO	—

CURSO

id	nome	turno	unidade
1	Análise e Desenvolvimento de Sistemas	—	FATEC Zona Leste

PERIODO_LETIVO

id	ano	semestre	data_inicio	data_fim
1	2026	1	—	—

GRADE

id	professor_id	periodo_letivo_id	semanal	mensal	total
1	1	1	6.00 (ilustrativo)	— (fórmula A definir)	—

DISCIPLINA_DA_GRADE

id	grade_id	disciplina_id	curso_id	semestre_curso	periodo	quantidade_aulas	op	status
1	1	1	1	—	Tarde	6	—	MINISTRANDO

AULA_DA_GRADE — modelo por bloco contínuo (seção 5.7): o horário 13:00–18:20 da terça-feira vira **uma única linha**; a expansão em slots de 50 minutos (1º a 6º da tarde) é feita pelo serviço na geração da Folha, não no banco:

id	disciplina_grade_id	dia_semana	hora_inicio	hora_fim
1	1	2 (terça)	13:00	18:20

Isso é validado contra `quantidade_aulas = 6` pela contagem de slots ocupados na tabela de turnos (seção 5.7), não por uma divisão simples de minutos.

USUARIO (exemplo)

id	nome	login	senha_hash	perfil	ativo
1	Usuário de exemplo	exemplo.resp	\$2b\$... (hash, nunca a senha)	RESPONSAVEL	true

29. Exemplo de relacionamento completo

```

Professor
  ↓
Grade
  ↓
Disciplina da Grade
  ↓
Disciplina
  ↓
Curso
  ↓
Aulas

```

Com os dados de exemplo:

```

Alexandre Falcão Sanseverino
  ↓
Grade 2026/1
  ↓
MAG – Álgebra Linear
  ↓
MAG004
  ↓
ADS
  ↓
Terça-feira
  ↓
13:00–18:20

```

Como cada informação é obtida pelos relacionamentos:

Informação	Caminho
Alexandre Falcão Sanseverino	<code>GRADE.professor_id</code> → <code>PROFESSOR.nome</code>
2026/1	<code>GRADE.periodo_letivo_id</code> → <code>PERIODO_LETIVO.ano / semestre</code>
Disciplina na Grade	<code>DISCIPLINA_DA_GRADE.grade_id = GRADE.id</code>
MAG / MAG004 / Álgebra Linear	<code>DISCIPLINA_DA_GRADE.disciplina_id</code> → <code>DISCIPLINA.sigla / codigo / nome</code>
ADS	<code>DISCIPLINA_DA_GRADE.curso_id</code> → <code>CURSO.nome</code>
Terça-feira, 13:00–18:20	<code>AULA_DA_GRADE.disciplina_grade_id = DISCIPLINA_DA_GRADE.id</code> → <code>dia_semana</code> , <code>hora_inicio</code> da 1ª aula, <code>hora_fim</code> da 6ª

30. Consulta para a Folha

O backend precisa obter: professor, grade, período, disciplina, sigla, curso, dia da semana, horário e blocos. **Não é necessária uma tabela específica da Folha**: os dados vêm dos relacionamentos existentes.

Passo 1 — validar que a Grade pertence ao professor (resulta em 422 se não pertencer):

```
SELECT 1
FROM grade
WHERE id = :gradeId AND professor_id = :professorId;
```

Passo 2 — carregar os dados (somente leitura), exemplo conceitual com os nomes reais das colunas:

```
SELECT p.nome           AS professor,
       p.matricula,
       p.regime_contrato,
       g.id              AS grade_id,
       pl.ano, pl.semestre,
       d.codigo, d.sigla, d.nome AS disciplina,
       c.nome            AS curso,
       dg.periodo, dg.semestre_curso,
       a.dia_semana, a.hora_inicio, a.hora_fim
FROM grade g
JOIN professor      p ON p.id = g.professor_id
JOIN periodo_letivo pl ON pl.id = g.periodo_letivo_id
JOIN disciplina_da_grade dg ON dg.grade_id = g.id
JOIN disciplina      d ON d.id = dg.disciplina_id
JOIN curso           c ON c.id = dg.curso_id
JOIN aula_da_grade   a ON a.disciplina_grade_id = dg.id
WHERE g.id = :gradeId
ORDER BY a.dia_semana, a.hora_inicio;
```

Na prática, isso vira um repositório JPA (`JOIN FETCH` ou projeção/DTO) com parâmetros nomeados. Se `codigo` for aprovado em `PROFESSOR`, ele entra no `SELECT`. Um `LEFT JOIN` em `aula_da_grade` evita "perder" disciplinas ainda sem aulas cadastradas.

O que NÃO vem do SQL (é calculado no `FolhaFrequenciaService` / `GradeService`):

Cálculo	Detalhe
Slot do modelo (bloco)	O conversor de horários usa o <code>hora_inicio</code> e escolhe o slot mais próximo; empate → o posterior. No exemplo: 13:00 → 1º, 13:50 → 2º, 14:50 → 3º, 15:40 → 4º, 16:40 → 5º e 17:30 → 6º da tarde, todos preenchidos com <code>MAG</code> .
Datas, dias da semana, domingos	A partir do mês/ano informado.
Dias sem aula	Do Calendário Acadêmico (A definir , seção 18).
Conflitos de slot	Duas aulas no mesmo dia/slot geram erro descritivo (D-14).

O resultado é organizado no `FolhaModel`, fronteira entre negócio e documento. O `DocxService` recebe só esse objeto.

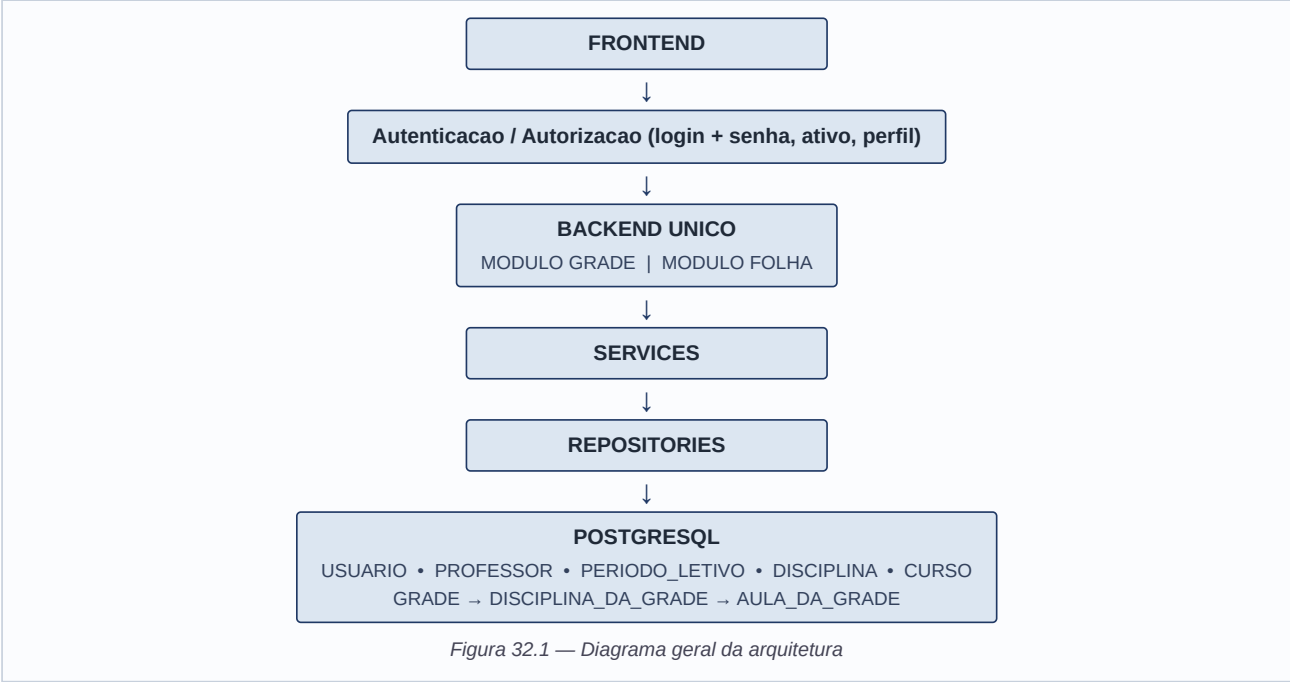
31. Separação de responsabilidades

Componente	Responsabilidade
Banco (PostgreSQL)	Armazena os dados estruturados.
Sistema de Grade	Gerencia e organiza as Grades.
Sistema de Folha	Lê os dados da Grade e prepara o documento.
Apache POI	Manipula o DOCX (sempre sobre uma cópia do modelo).

Componente	Responsabilidade
Calendário	Fornecer informações acadêmicas relacionadas às datas.
USUARIO / segurança	Autentica e autoriza o acesso ao backend.
Controle de Ponto	Não faz parte da responsabilidade atual deste sistema.

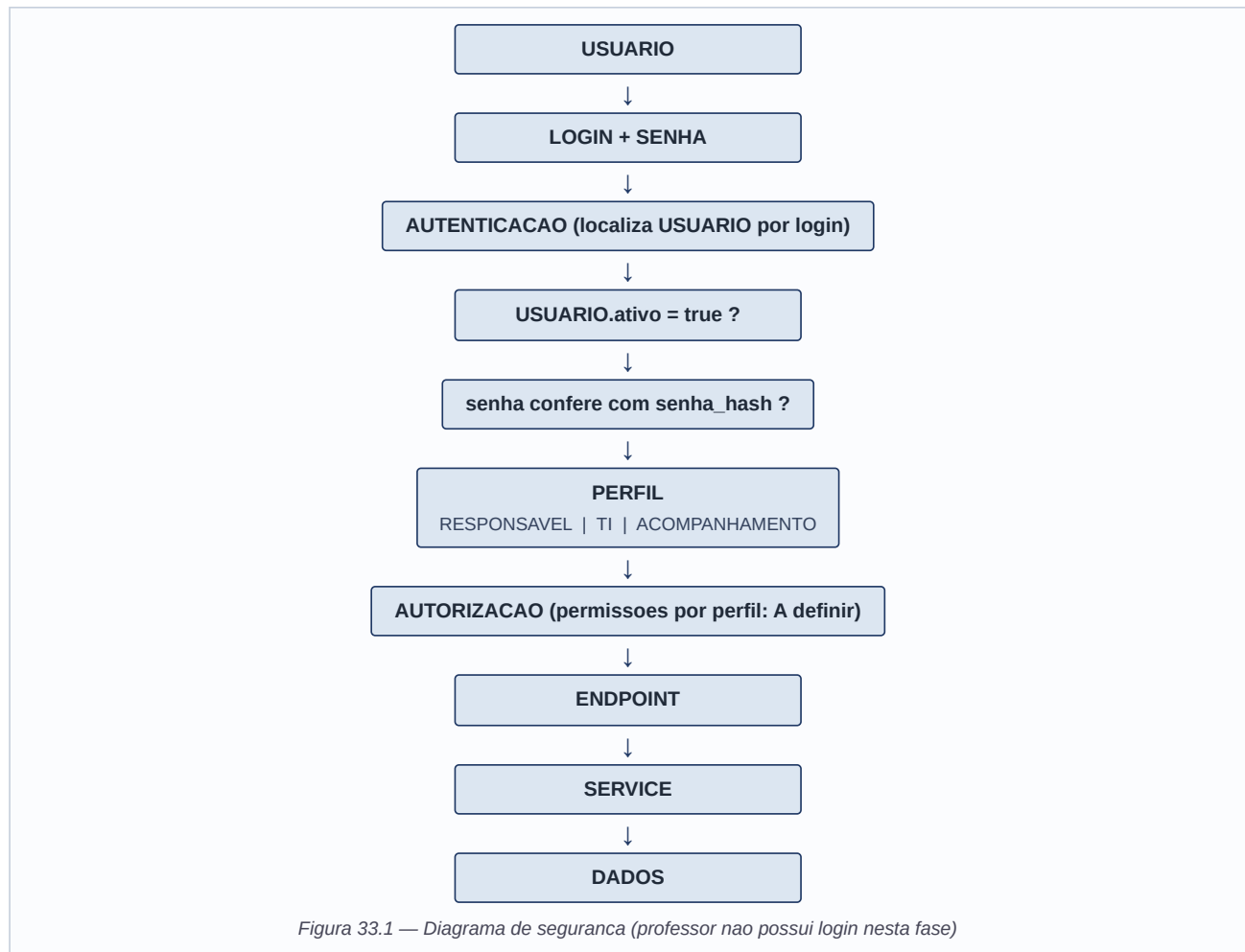
Separação conceitual: GRADE HORÁRIA ≠ FOLHA DE FREQUÊNCIA ≠ PRESENÇA/PONTO.

32. Diagrama geral da arquitetura



O `USUARIO` é responsável pela autenticação e autorização do acesso ao backend. As tabelas acadêmicas são acessadas pelos dois módulos, por meio de Services e Repositories.

33. Diagrama de segurança



34. Critérios de aceite do banco

Estrutura

- ☐ As 8 entidades existem: PROFESSOR, CURSO, DISCIPLINA, PERIODO_LETIVO, GRADE, DISCIPLINA_DA_GRADE, AULA_DA_GRADE, USUARIO.
- ☐ Todas possuem PK (id BIGINT).
- ☐ Os relacionamentos possuem FK (as 6 do quadro da seção 8) e as cardinalidades estão documentadas.
- ☐ O PostgreSQL é utilizado.
- ☐ Não existe duplicação desnecessária de dados.
- ☐ Não existe tabela/registro de presença criado sem requisito.

Regras do modelo

- ☐ Um professor pode possuir várias Grades.
- ☐ Uma Grade pertence a um único período letivo.
- ☐ Uma Grade possui várias associações de disciplinas.
- ☐ Uma disciplina pode aparecer em várias Grades.
- ☐ O curso pode estar associado às disciplinas da Grade.
- ☐ Uma disciplina da Grade possui aulas.
- ☐ A carga horária pertence à Grade.

- ☐ `codigo` e `sigla` da disciplina são campos distintos.
- ☐ Todas as FKs usam `RESTRICT`; o histórico acadêmico não é apagado indevidamente.

Usuários e segurança

- ☐ O usuário possui `login` único.
- ☐ A senha é armazenada **apenas como hash**.
- ☐ O usuário pode ser bloqueado por `ativo`, sem ser excluído.
- ☐ Não existe login de professor.
- ☐ Credenciais do banco não estão no código-fonte nem no Git.

Integração

- ☐ Grade e Folha utilizam o mesmo banco e o mesmo backend.
- ☐ A Folha consegue obter todos os dados necessários da Grade por meio de relacionamentos.
- ☐ A geração da Folha **não altera** a Grade.

35. Backlog / Issues do banco

Numeração `DB-xx` para não colidir com as issues numeradas dos backlogs de backend e da Folha. Há sobreposição entre os três backlogs (por exemplo, "integração com o banco existente" e "mapeamento de entidades"); recomenda-se **consolidá-los** ao criar as issues no GitHub Projects, evitando trabalho duplicado. Onde o banco já existe, as issues de criação de tabela funcionam como **verificação/adequação** do que já há.

DB-01 — Criação do banco PostgreSQL

- **Objetivo:** ter um banco único compartilhado por Grade e Folha.
- **Descrição:** provisionar/verificar a instância e o esquema (Docker Compose se necessário); criar o usuário de aplicação com menor privilégio; credenciais por variáveis de ambiente.
- **Critérios de aceite:** a aplicação conecta; nenhuma credencial no Git; `ddl-auto=validate`.
- **Dependências:** —
- **Prioridade:** Alta

DB-02 — Criação da tabela PROFESSOR

- **Objetivo:** persistir o cadastro de professores.
- **Descrição:** campos da seção 5.1; `UNIQUE` em `cpf` e `matricula`. Tratar `codigo` e `categoria` só após a decisão.
- **Critérios de aceite:** PK e `UNIQUE`s ativos; sem coluna nova não confirmada.
- **Dependências:** DB-01, DB-14
- **Prioridade:** Alta

DB-03 — Criação da tabela CURSO

- **Objetivo:** persistir os cursos.
- **Descrição:** campos da seção 5.2. **Sem** `codigo` até a definição oficial.
- **Critérios de aceite:** tabela criada com PK; pendência do código registrada.
- **Dependências:** DB-01, DB-14
- **Prioridade:** Alta

DB-04 — Criação da tabela DISCIPLINA

- **Objetivo:** persistir o cadastro geral de disciplinas.
- **Descrição:** `codigo`, `sigla` e `nome`; sem `curso_id`.
- **Critérios de aceite:** `codigo` e `sigla` são campos distintos; `UNIQUE(codigo)` só se confirmado.
- **Dependências:** DB-01, DB-14

- **Prioridade:** Alta

DB-05 — Criação da tabela PERIODO_LETIVO

- **Objetivo:** identificar o período de cada Grade e preservar o histórico.
- **Descrição:** `semestre`, `ano`, `data_inicio`, `data_fim`; `CHECK` de semestre e de datas.
- **Critérios de aceite:** semestre fora de {1, 2} e `data_inicio ≥ data_fim` são rejeitados.
- **Dependências:** DB-01, DB-14
- **Prioridade:** Alta

DB-06 — Criação da tabela GRADE

- **Objetivo:** persistir a Grade (professor + período) com a carga horária.
- **Descrição:** campos da seção 5.5, com FK para `PROFESSOR` e `PERIODO_LETIVO`.
- **Critérios de aceite:** Grade sem professor ou período é rejeitada; um professor pode ter várias Grades; carga horária na própria Grade.
- **Dependências:** DB-02, DB-05
- **Prioridade:** Alta

DB-07 — Criação da tabela DISCIPLINA_DA_GRADE

- **Objetivo:** associar disciplina, Grade e curso.
- **Descrição:** campos da seção 5.6, com FKs para `GRADE`, `DISCIPLINA` e `CURSO`; `CHECK(quantidade_aulas > 0)`.
- **Critérios de aceite:** a mesma disciplina pode entrar em várias Grades e cursos; `quantidade_aulas ≤ 0` é rejeitada.
- **Dependências:** DB-03, DB-04, DB-06
- **Prioridade:** Alta

DB-08 — Criação da tabela AULA_DA_GRADE

- **Objetivo:** persistir dia e horário das aulas.
- **Descrição:** `dia_semana`, `hora_inicio`, `hora_fim` (TIME), FK para `DISCIPLINA_DA_GRADE`; `CHECK(hora_fim > hora_inicio)`. Definir a codificação de `dia_semana`.
- **Critérios de aceite:** horário final ≤ inicial é rejeitado; codificação de `dia_semana` documentada.
- **Dependências:** DB-07
- **Prioridade:** Alta

DB-09 — Criação da tabela USUARIO

- **Objetivo:** suportar o controle de acesso.
- **Descrição:** campos da seção 5.8; `UNIQUE(login)`; `senha_hash`; `ativo`; `created_at` / `updated_at`.
- **Critérios de aceite:** login duplicado é rejeitado; não existe coluna de senha em texto puro.
- **Dependências:** DB-01, DB-14
- **Prioridade:** Alta

DB-10 — Criação das FKs

- **Objetivo:** garantir a integridade referencial.
- **Descrição:** as 6 FKs da seção 8, todas com `ON DELETE RESTRICT`. Normalmente entregues junto das migrations das tabelas.
- **Critérios de aceite:** inserir com FK inexistente e excluir "pai" com "filhos" falham.
- **Dependências:** DB-02 a DB-08
- **Prioridade:** Alta

DB-11 — Criação dos índices

- **Objetivo:** dar desempenho às consultas da Grade e da Folha.
- **Descrição:** índices da seção 24 (FKs); não indexar colunas de baixa cardinalidade.
- **Critérios de aceite:** índices criados; `EXPLAIN` da consulta da seção 30 usa os índices em volume de teste.
- **Dependências:** DB-10
- **Prioridade:** Média

DB-12 — Criação das constraints

- **Objetivo:** proteger as regras de negócio no banco.
- **Descrição:** `UNIQUE`, `CHECK` e `NOT NULL` da seção 25. Aplicar só as **Definidas** e as **Sugeridas** aprovadas; **não** aplicar as **A definir**.
- **Critérios de aceite:** cada constraint tem sua regra citada; dados inválidos são rejeitados.
- **Dependências:** DB-02 a DB-09
- **Prioridade:** Alta

DB-13 — Configuração do JPA

- **Objetivo:** mapear as tabelas em entidades.
- **Descrição:** entidades e repositórios das 8 tabelas (seção 26); `ddl-auto=validate`; `LAZY`; sem cascade de remoção automático.
- **Critérios de aceite:** a aplicação sobe com `validate`; consultas simples funcionam; nenhuma tabela é criada pelo Hibernate.
- **Dependências:** DB-02 a DB-10
- **Prioridade:** Alta

DB-14 — Configuração das migrations

- **Objetivo:** versionar o esquema do banco.
- **Descrição:** avaliar e adotar Flyway (ou equivalente); definir *baseline* se o banco já existe; estabelecer a convenção `V{n}__descricao.sql`.
- **Critérios de aceite:** um ambiente vazio é reconstruído só pelas migrations; migrations aplicadas são imutáveis.
- **Dependências:** DB-01
- **Prioridade:** Alta

DB-15 — Implementação da autenticação

- **Objetivo:** permitir login seguro.
- **Descrição:** fluxo da seção 20 (login → `ativo` → hash → perfil), com `PasswordEncoder`.
- **Critérios de aceite:** usuário inativo não entra; senha nunca em texto puro nem em log; mensagem de erro genérica.
- **Dependências:** DB-09, DB-13
- **Prioridade:** Média (**Alta** antes de expor a API fora da rede interna)

DB-16 — Implementação da autorização

- **Objetivo:** restringir operações por perfil.
- **Descrição:** aplicar a matriz da seção 21 **depois de validada**; itens "A definir" ficam bloqueados por padrão.
- **Critérios de aceite:** cada perfil acessa apenas o que foi aprovado; testes por endpoint.
- **Dependências:** DB-15; definição oficial das permissões
- **Prioridade:** Média

DB-17 — Integração Grade/Folha

- **Objetivo:** a Folha ler os dados da Grade sem duplicá-los.

- **Descrição:** consulta da seção 30 no `FolhaFrequenciaService`, em leitura; validação de que a Grade pertence ao professor.
- **Critérios de aceite:** todos os dados do documento são obtidos por relacionamento; a geração não altera a Grade (`updated_at` inalterado).
- **Dependências:** DB-06 a DB-08, DB-11, DB-13
- **Prioridade:** Alta

DB-18 — Testes do banco

- **Objetivo:** verificar estrutura e constraints.
- **Descrição:** testes com PostgreSQL real (Testcontainers) para PK, FK, `UNIQUE`, `CHECK`, `RESTRICT` e migrations.
- **Critérios de aceite:** cada constraint tem ao menos um teste positivo e um negativo.
- **Dependências:** DB-10, DB-12, DB-14
- **Prioridade:** Média

DB-19 — Testes de integração

- **Objetivo:** verificar o fluxo ponta a ponta.
- **Descrição:** `professor → grade → disciplinas → aulas → FolhaModel`; login com usuário ativo/inativo; Folha somente leitura.
- **Critérios de aceite:** fluxo completo verde; template inalterado; nenhum registro de presença criado.
- **Dependências:** DB-13, DB-15, DB-17
- **Prioridade:** Média

36. Estrutura final do modelo

```
PROFESSOR
CURSO
DISCIPLINA
PERIODO_LETIVO
GRADE
DISCIPLINA_DA_GRADE
AULA_DA_GRADE
USUARIO
```

Relação principal:

```
PROFESSOR
↓
GRADE
↓
DISCIPLINA_DA_GRADE
↓
AULA_DA_GRADE
```

Relações complementares:

```
GRADE → PERIODO_LETIVO
DISCIPLINA_DA_GRADE → DISCIPLINA
DISCIPLINA_DA_GRADE → CURSO
USUARIO → CONTROLE DE ACESSO
```

- A carga horária fica armazenada na própria **GRADE**.
- **GRADE** e **FOLHA** pertencem ao mesmo sistema, ao mesmo backend e ao mesmo banco.

37. Regras importantes

1. Grade e Folha **não** são dois sistemas independentes; são módulos do mesmo sistema/backend.
2. Ambos usam o mesmo PostgreSQL.
3. A Grade é a fonte das informações acadêmicas usadas pela Folha.
4. A Folha não duplica desnecessariamente os dados da Grade e **não altera** a Grade ao gerar o documento.
5. Grade Horária **não é** presença; Folha de Frequência **não é** a mesma coisa que Grade.
6. Presença/ponto **não** é criado no banco sem requisito específico.
7. `USUARIO` controla o acesso; não existe login de professor nesta fase.
8. A senha nunca é armazenada em texto puro (`senha_hash`); `ativo` bloqueia o acesso; `created_at` / `updated_at` auxiliam no controle.
9. Perfis previstos: `RESPONSAVEL` , `TI` , `ACOMPANHAMENTO` . Permissões não oficiais estão marcadas como **A definir**.
10. O banco preserva a integridade referencial; o histórico acadêmico **não** é excluído automaticamente.
11. Não foram criadas tabelas, entidades ou campos obrigatórios sem justificativa no domínio.
12. Requisitos **[Definido]** e sugestões **[Sugestão]** estão diferenciados ao longo do texto.
13. MER e DER representam os relacionamentos reais; `USUARIO` aparece na camada de segurança, separado das entidades acadêmicas.
14. A estrutura está preparada para a integração com o módulo de Folha de Frequência.

38. Divergências entre as fontes e decisões adotadas

38.1 Divergências de nome, tipo e estrutura

Onde o prompt e o modelo existente divergem, foi adotado o **modelo existente**, por ser o documento técnico do banco. Todos os itens abaixo devem ser confirmados pelo time.

#	Item	Prompt de requisitos	Modelo existente / outra fonte	Adotado aqui
1	Regime do professor	<code>regime_juridico</code>	<code>regime_contrato</code> (o placeholder da Folha, <code>{{RJ}}</code> , aponta para ele)	Os dois campos , lado a lado (decisão desta rodada) — ver nota na seção 5.1
2	Carga horária	<code>carga_horaria</code> <code>INTEGER</code>	<code>semanal</code> , <code>mensal</code> , <code>total</code> (<code>NUMERIC(6,2)</code>) na <code>GRADE</code>	Os três campos existentes
3	FK da aula	<code>disciplina_da_grade_id</code>	<code>disciplina_grade_id</code>	<code>disciplina_grade_id</code> (renomear exigiria migration e ajuste do JPA)
4	<code>dia_semana</code>	Texto (<code>TERCA</code>)	<code>SMALLINT</code>	<code>SMALLINT</code> ; codificação A definir
5	<code>ano / semestre</code>	<code>INTEGER</code>	<code>SMALLINT</code>	<code>SMALLINT</code>
6	Código do curso	<code>codigo</code> <code>VARCHAR(...)</code> (exemplo)	"O código oficial do curso ainda não foi definido"	Sem código em <code>CURSO</code>
7	Bloco 13:00–18:20	Uma única aula/bloco	Uma linha por aula de 50 min	Bloco único por linha (decisão desta rodada, seção 5.7) — resolve D-07b e esclarece D-08; expansão em slots feita no serviço, não no banco
8	Código do professor	Não consta	<code>{{COD}}</code> da Folha precisa dele	<code>PROFESSOR.codigo</code> adotado (decisão desta rodada, coluna nulável)
9	Nome do serviço	<code>FolhaPontoService</code>	<code>FolhaFrequenciaService</code> (D-01)	<code>FolhaFrequenciaService</code>
10	Autenticação	Definida (login/senha/perfis)	Folha D-16: "não definida / fora do escopo agora"	Modelo definido; implementação nas issues DB-15/DB-16

#	Item	Prompt de requisitos	Modelo existente / outra fonte	Adotado aqui
11	Rota da Folha	—	Backend: <code>/api/folha-frequencia/gerar</code> ; Folha: <code>/api/folhas-frequencia/preview</code> e <code>/gerar</code>	Unificar (recomendado: <code>/api/folhas-frequencia</code>)
12	matricula	Obrigatória e única (regra de negócio do banco)	Folha D-10: pode estar vazia (exibe "—")	NOT NULL no banco; o documento exibe "—" para valores ausentes
13	Turma e Atividade	—	O texto do banco menciona "Atividade da Grade", mas ela não está entre as 8 entidades; <code>TURMA</code> é pendência	Não criadas

38.2 Pendências consolidadas ("A definir")

#	Pendência	Impacto no banco
1	Código oficial de curso e de disciplina	Colunas/ <code>UNIQUE</code>
2	<code>PROFESSOR.codigo</code> e <code>categoria</code>	Novas colunas (aditivas)
3	Significado, faixa e unicidade do <code>op</code>	<code>CHECK</code> / <code>UNIQUE</code>
4	Atividades V/D	Possível nova entidade
5	Fórmula de carga horária mensal	Regra de cálculo em <code>GRADE</code>
6	Valores oficiais de <code>status</code> , <code>titulacao</code> , <code>regime_contrato</code> , <code>perfil</code> , <code>periodo</code>	<code>CHECK</code> de domínio
7	Conceito oficial de <code>TURMA</code>	Nova entidade + FK
8	Codificação de <code>dia_semana</code>	<code>CHECK</code> + mapeamento Java
9	Limite de "uma Grade por professor/período"	<code>UNIQUE</code> composto
10	Faixa válida do <code>ano</code>	<code>CHECK</code>
11	Validação final da FATEC sobre o modelo por bloco em <code>AULA_DA_GRADE</code> (proposto nesta rodada, seção 5.7)	Convenção de uso da tabela, sem mudança de esquema
12	Exclusão de <code>AULA_DA_GRADE</code> (<code>RESTRICT</code> × <code>CASCADE</code>) e regras de manutenção do histórico	Regra de exclusão
17	Confirmar se <code>regime_juridico</code> e <code>regime_contrato</code> são a mesma informação (então um dos dois pode ser removido depois)	<code>PROFESSOR</code>
13	Armazenamento do Calendário Acadêmico (D-03)	Possível nova entidade
14	Permissões exatas por perfil	Autorização
15	Algoritmo de hash, sessão/token e política de backup	Segurança
16	Integração com AxioDB ou outras fontes institucionais	Origem dos dados

39. Verificação final de consistência

Verificação	Resultado
<code>PROFESSOR 1:N GRADE</code>	✓ FK <code>grade.professor_id</code> (seções 8, 9, 11, 12)
<code>PERIODO_LETIVO 1:N GRADE</code>	✓ FK <code>grade.periodo_letivo_id</code>
<code>GRADE 1:N DISCIPLINA_DA_GRADE</code>	✓ FK <code>disciplina_da_grade.grade_id</code>
<code>DISCIPLINA 1:N DISCIPLINA_DA_GRADE</code>	✓ FK <code>disciplina_da_grade.disciplina_id</code> (a forma original "N:1" foi preservada e explicada)
<code>CURSO 1:N DISCIPLINA_DA_GRADE</code>	✓ FK <code>disciplina_da_grade.curso_id</code>

Verificação	Resultado
DISCIPLINA_DA_GRADE 1:N AULA_DA_GRADE	✓ FK aula_da_grade.disciplina_grade_id
USUARIO controla o acesso	✓ Sem FK para as entidades acadêmicas; isolado no MER/DER
Estrutura final com as 8 entidades	✓ Seções 4, 10 e 36; MER e DER contêm as 8
Relação principal PROFESSOR → GRADE → DISCIPLINA_DA_GRADE → AULA_DA_GRADE	✓ Seção 10
GRADE → PERIODO_LETIVO, DISCIPLINA_DA_GRADE → DISCIPLINA e → CURSO	✓ Seção 10
Carga horária na própria GRADE	✓ semanal, mensal, total
senha_hash, ativo, created_at, updated_at em USUARIO	✓ Seção 5.8
Não existe tabela de presença/ponto, nem tabela desnecessária	✓ Seção 4
Grade e Folha no mesmo sistema, backend e banco	✓ Seções 1, 2 e 16
A Folha só lê a Grade	✓ Seção 17
Pendências marcadas como "A definir"	✓ Seção 38.2

GRADE HORÁRIA ≠ FOLHA DE FREQUÊNCIA ≠ PRESENÇA/PONTO — separação mantida em todo o documento.

Apêndice A — Script DDL de referência

Proposta para conferência (**não é uma migration pronta**). Marcações nos comentários: **[D]** = definido nas fontes; **[S]** = sugestão técnica; **[A definir]** = pendente (não aplicado ou deixado nulaável). Se o banco já existe, compare com o esquema real antes de qualquer alteração.

```

-- =====
-- Camada acadêmica
-- =====
CREATE TABLE professor (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome        VARCHAR(150) NOT NULL,                -- [D] exigido pela Folha
    codigo      VARCHAR(30),                          -- [D-decisão] {{COD}} da Folha; nulável
    cpf         VARCHAR(11) NOT NULL,                 -- [D]
    matricula   VARCHAR(30) NOT NULL,                 -- [D]
    regime_contrato VARCHAR(20) NOT NULL,             -- [D] nome conforme o modelo existente
    regime_juridico VARCHAR(20),                     -- [D-decisão] nome conforme o prompt; mantido
    status      VARCHAR(20) NOT NULL,                 -- [D]
    titulacao   VARCHAR(30),                          -- [A definir] obrigatoriedade
    CONSTRAINT uk_professor_cpf          UNIQUE (cpf), -- [D]
    CONSTRAINT uk_professor_matricula    UNIQUE (matricula) -- [D]
    -- categoria: avaliado e NÃO incorporado (D-09, sem placeholder confirmado no DOCX)
);

CREATE TABLE curso (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome        VARCHAR(150) NOT NULL,                -- [S]
    turno       VARCHAR(20),                          -- [A definir]
    unidade     VARCHAR(100)                          -- [A definir]
    -- codigo -> [A definir] (código oficial ainda não definido)
);

CREATE TABLE disciplina (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    codigo      VARCHAR(30) NOT NULL,                 -- [S] código oficial (ex.: MAG004)
    sigla       VARCHAR(20) NOT NULL,                 -- [S] sigla curta (ex.: MAG)
    nome        VARCHAR(150) NOT NULL                 -- [S]
    -- UNIQUE (codigo) -> [A definir]
);

CREATE TABLE periodo_letivo (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    semestre    SMALLINT NOT NULL,                   -- [D]
    ano         SMALLINT NOT NULL,                   -- [D] faixa: [A definir]
    data_inicio DATE,                                -- [A definir]
    data_fim    DATE,                                -- [A definir]
    CONSTRAINT ck_periodo_semestre CHECK (semestre IN (1, 2)), -- [D]
    CONSTRAINT ck_periodo_datas    CHECK (data_inicio < data_fim), -- [D] (nulos passados)
    CONSTRAINT uk_periodo_ano_sem  UNIQUE (ano, semestre) -- [S]
);

CREATE TABLE grade (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    professor_id BIGINT NOT NULL,                    -- [D]
    periodo_letivo_id BIGINT NOT NULL,                -- [D]
    semanal     NUMERIC(6,2),                        -- [A definir] obrigatoriedade
    mensal      NUMERIC(6,2),                        -- [A definir] fórmula oficial
    total       NUMERIC(6,2),                        -- [A definir]
    created_at  TIMESTAMP NOT NULL DEFAULT now(),    -- [S] default
    updated_at  TIMESTAMP NOT NULL DEFAULT now(),    -- [S] default
    CONSTRAINT fk_grade_professor FOREIGN KEY (professor_id)
        REFERENCES professor (id) ON DELETE RESTRICT,
    CONSTRAINT fk_grade_periodo FOREIGN KEY (periodo_letivo_id)
        REFERENCES periodo_letivo (id) ON DELETE RESTRICT,
    CONSTRAINT ck_grade_carga CHECK (
        COALESCE(semanal, 0) >= 0 AND COALESCE(mensal, 0) >= 0 AND COALESCE(total, 0) >= 0 -- [S]
    )
    -- UNIQUE (professor_id, periodo_letivo_id) -> [A definir]
);

CREATE TABLE disciplina_da_grade (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    grade_id    BIGINT NOT NULL,                    -- [D]
    disciplina_id BIGINT NOT NULL,                  -- [D]
    curso_id    BIGINT NOT NULL,                    -- [S] a relação com o curso é definida
    semestre_curso SMALLINT,                        -- [A definir]

```

```

    periodo          VARCHAR(20),                -- [A definir] Manhã | Tarde | Noite
    quantidade_aulas  SMALLINT NOT NULL,          -- [D]
    op                INTEGER,                    -- [A definir]
    status            VARCHAR(20) NOT NULL,       -- [D] ex.: MINISTRANDO
    CONSTRAINT fk_dg_grade FOREIGN KEY (grade_id) REFERENCES grade (id) ON DELETE RESTRICT
    CONSTRAINT fk_dg_disciplina FOREIGN KEY (disciplina_id) REFERENCES disciplina (id) ON DELETE RESTRICT
    CONSTRAINT fk_dg_curso FOREIGN KEY (curso_id) REFERENCES curso (id) ON DELETE RESTRICT
    CONSTRAINT ck_dg_qtd_aulas CHECK (quantidade_aulas > 0) -- [D]
);

CREATE TABLE aula_da_grade (
    id                BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    disciplina_grade_id BIGINT NOT NULL,          -- [D] nome conforme o modelo existente
    dia_semana        SMALLINT NOT NULL,         -- [S] codificação: [A definir]
    hora_inicio        TIME NOT NULL,             -- [D]
    hora_fim           TIME NOT NULL,             -- [D]
    CONSTRAINT fk_aula_dg FOREIGN KEY (disciplina_grade_id)
        REFERENCES disciplina_da_grade (id) ON DELETE RESTRICT, -- CASCADE: [A definir] (seção 14.1)
    CONSTRAINT ck_aula_horario CHECK (hora_fim > hora_inicio) -- [D]
    -- CHECK (dia_semana BETWEEN 1 AND 7) -> [A definir]
);

-- =====
-- Camada de segurança (sem FK para a camada acadêmica)
-- =====

CREATE TABLE usuario (
    id                BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome              VARCHAR(150),               -- [A definir]
    login             VARCHAR(50) NOT NULL,       -- [D]
    senha_hash        VARCHAR(255) NOT NULL,      -- [D] nunca a senha em texto puro
    perfil            VARCHAR(30) NOT NULL,       -- [D] RESPONSÁVEL | TI | ACOMPANHAMENTO
    ativo             BOOLEAN NOT NULL DEFAULT TRUE, -- [S] default
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(), -- [S] default
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(), -- [S] default
    CONSTRAINT uk_usuario_login UNIQUE (login)    -- [D]
    -- CHECK (perfil IN ('RESPONSÁVEL','TI','ACOMPANHAMENTO')) -> [A definir]
);

-- =====
-- Índices em FKs [S] (PK e UNIQUE já criam seus índices)
-- =====

CREATE INDEX idx_grade_professor_id ON grade (professor_id);
CREATE INDEX idx_grade_periodo_letivo_id ON grade (periodo_letivo_id);
CREATE INDEX idx_dg_grade_id ON disciplina_da_grade (grade_id);
CREATE INDEX idx_dg_disciplina_id ON disciplina_da_grade (disciplina_id);
CREATE INDEX idx_dg_curso_id ON disciplina_da_grade (curso_id);
CREATE INDEX idx_aula_disciplina_grade_id ON aula_da_grade (disciplina_grade_id);

```